

09:00-09:50 (쉬는 시간 11:30-12:00)

코드 리뷰 기준과 Severity

Finding · Severity · review_rubric.md

진행

코드 리뷰 기준과 Severity
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

AGENTS.md
src/course_services/contracts.py

확인할 결과

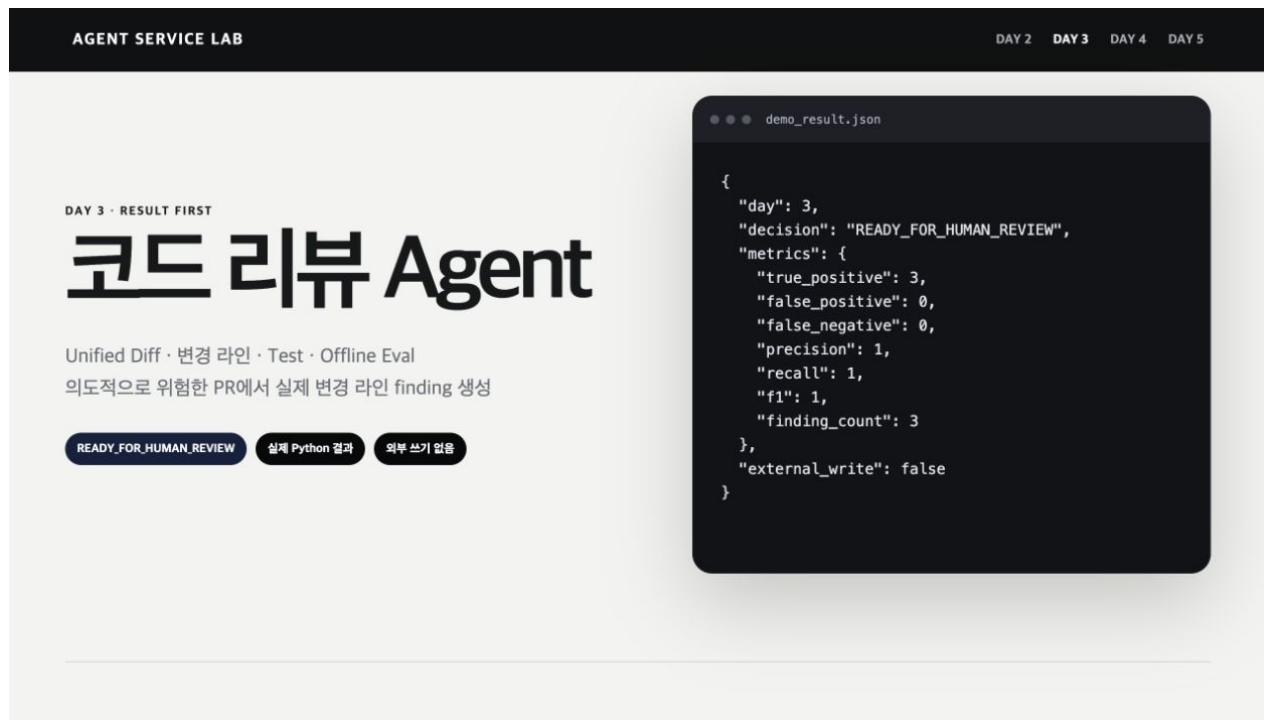
review_rubric.md

Review Intelligence Service 완성 화면

스타일 지적이 많아질수록 중요한
correctness 오류가 묻힌다.

입력 계약

- 핵심 처리
- 실패 경계
- 사람 판단
- 평가·실행 증거



재생 파일 · assets/demo-videos/day3_service_teaser.gif

오늘 8차시는 이 화면을 현재 저장소에서 다시 만드는 과정입니다.

3일차 전체 시간표

3시간 블록(30분 휴식) · 2시간 블록(20분 휴식) · 점심 14:00-15:00 · 3시간 블록(30분 휴식)

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	코드 리뷰 기준과 Severity	강의 12 시연 10 Codex 제작 23 확인 5분	review_rubric.md
09:50-10:40	2차시	Unified Diff와 Line Mapping	강의 12 시연 10 Codex 제작 23 확인 5분	parsed_hunks.json
10:40-11:30	3차시	리뷰 Context Pack	강의 12 시연 10 Codex 제작 23 확인 5분	context_pack.json
12:00-12:50	4차시	ReviewFinding 결과 계약	강의 12 시연 10 Codex 제작 23 확인 5분	review_report.json
12:50-13:40	5차시	Prompt Baseline과 의도적 Bug	강의 12 시연 10 Codex 제작 23 확인 5분	baseline_review.json
15:00-15:50	6차시	정적 분석·LLM Hybrid Review	강의 12 시연 10 Codex 제작 23 확인 5분	hybrid_review.json
15:50-16:40	7차시	Precision·Recall 기반 평가	강의 12 시연 10 Codex 제작 23 확인 5분	review_eval.json
16:40-17:30	8차시	오류 분석과 회귀 방지	강의 12 시연 10 Codex 제작 23 확인 5분	day3_review_report.md

11:30-12:00 쉬는 시간 · 13:40-14:00 쉬는 시간 · 14:00-15:00 점심시간 · 17:30-18:00 쉬는 시간

코드 리뷰 기준과 Severity 학습 결과

- 01 — P0~P3 severity와 actionable finding 기준을 합의한다.
- 02 — 모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다. 왜 위험한지 설명한다.
- 03 — review_rubric.md에서 정상과 실패 상태를 직접 확인한다.

코드 리뷰 기준과 Severity 용어

용어	이 수업에서 쓰는 뜻
Finding	한 가지 문제와 교정 제안
Severity	사용자 영향과 긴급도
False positive	문제가 아닌데 문제라고 한 결과
Confidence	근거에 대한 확신 정도

review_rubric.md 입출력

INPUT

AGENTS.md

OUTPUT

review_rubric.md

PROCESS

변경 확인 → 사용자 영향 → 재현 조건

VERIFY

한 finding에는 path·line·severity·근거·제안이 모두 있다.
변경과 무관한 파일 의견은 게시 후보에서 제외한다.

코드 리뷰 기준과 Severity 실행 흐름

01

변경 확인

02

사용자 영향

03

재현 조건

04

근거 라인

05

가장 작은 교정

마지막 판단은 `review_rubric.md`에서 사람이 확인합니다.

review_rubric.md 정상·실패 계약

정상	——	한 finding에는 path·line·severity·근거·제안이 모두 있다.
경계	——	변경과 무관한 파일 의견은 게시 후보에서 제외한다.
외부 쓰기	——	코드 리뷰 기준과 Severity: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·review_rubric.md

코드 리뷰 기준과 Severity 파일 지도

입력·fixture

AGENTS.md

구현 코드

src/course_services/contracts.py

검증·test

data/day3_review_cases/unsafe_pr.diff

따라하기 notebook

materials/day3/day3_service_lab.ipynb

코드 리뷰 기준과 Severity 개념·코드

Finding

한 가지 문제와 교정 제안

```
AGENTS.md
```

Severity

사용자 영향과 긴급도

```
src/course_services/contracts.py
```

False positive

문제가 아닌데 문제라고 한 결과

```
data/day3_review_cases/unsafe_pr.diff
```

Confidence

근거에 대한 확신 정도

```
materials/day3/day3_service_lab.ipynb
```

코드 리뷰 기준과 Severity 선택 기준

구분	역할	이번 차시의 판단 기준
Finding	변경 확인 단계의 입력 기준	결정론 test로 먼저 확인
Severity	사용자 영향 단계의 교체 경계	adapter 경계를 확인
False positive	재현 조건 단계의 운영 조건	비용·지연·권한을 확인
Confidence	근거 라인 단계의 판단 기록	사람 판단과 운영 기록을 확인

코드 리뷰 기준과 Severity 대표 실패

실패

모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다.

사용자 영향

모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다. 상태에서는 review_rubric.md을 운영 판단에 사용할 수 없다.

코드 리뷰 기준과 Severity 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

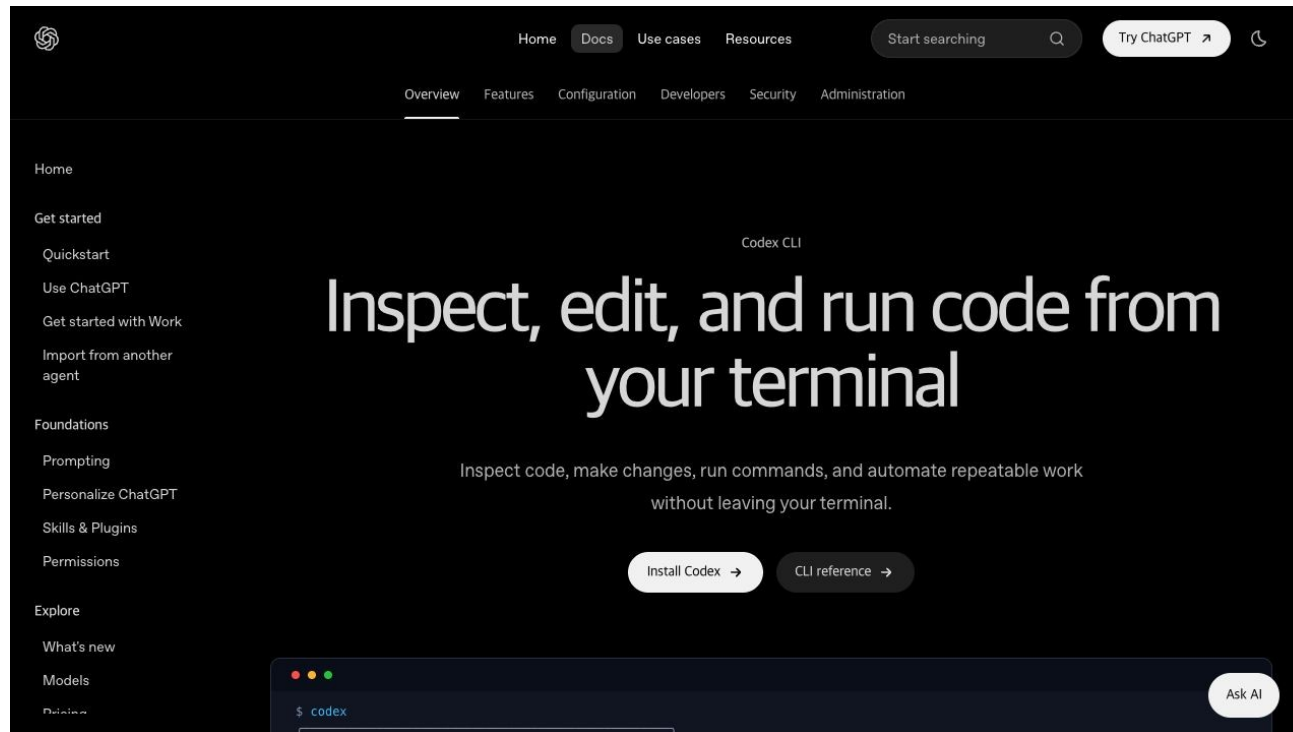
코드 리뷰 기준과 Severity 복구 경로

- 01 — 탐지: 변경과 무관한 파일 의견은 게시 후보에서 제외한다.
- 02 — 중단: 모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다.
- 03 — 복구: correctness·security·data loss·contract break에 집중하고 style은 linter에 맡긴다.
- 04 — 증거: review_rubric.md과 test 결과를 함께 남긴다.

코드 리뷰 기준과 Severity Codex 검증 루프

`src/course_services/contracts.py`의 책임을 찾고
`review_rubric.md` 계약을
`focused test`로 고정합니다.

코드 리뷰 기준과 Severity
저장소 이해 → 허용 범위 변경
→ `focused test` → `diff` 리뷰
→ 사람 merge



코드 리뷰 기준과 Severity Codex 작업 명세

목표

AGENTS.md 리뷰 기준을 읽고 바뀐 라인에만 finding을 작성하게 한다.

허용 경로

src/course_services/contracts.py
data/day3_review_cases/unsafe_pr.diff

완료 조건

한 finding에는 path·line·severity·근거·제안이 모두 있다.
변경과 무관한 파일 의견은 게시 후보에서 제외한다.

금지

코드 리뷰 기준과 Severity: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

코드 리뷰 기준과 Severity 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | 코드 리뷰 기준과 Severity 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 한 finding에는 path·line·severity·근거·제안이 모두 있다. |
| 03 | Claude review | —— | review_rubric.md 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 코드 리뷰 기준과 Severity의 두 LLM 의견과 test는 자동 승인이 아니다 |

코드 리뷰 기준과 Severity 정상 시연

01 — 열기: AGENTS.md

02 — 구현 확인: src/course_services/contracts.py

03 — 실행: python -m pytest -q tests/test_course_services.py -k maps_findings

04 — 성공 신호: 한 finding에는 path·line·severity·근거·제안이 모두 있다.

코드 리뷰 기준과 Severity 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k maps_findings
```

실행 전 확인

코드 리뷰 기준과 Severity: repository root · Python 3.12 · .env 미출력

01_review_rubric.json 실행 결과

```
{
  "required_fields": [
    "path",
    "line",
    "severity",
    "title",
    "body",
    "evidence",
    "suggestion",
    "confidence",
    "rule_id"
  ],
  "severity": {
    "P0": "즉시 악용·데이터 손실",
    "P1": "외부 상태·핵심 기능 위험",
    "P2": "복구·운영 품질 저하",
    "P3": "국소적 개선"
  },
  "excluded": [
    "style_only",
    "unrelated_file",
    "invented_runtime_result"
  ]
}
```

코드 리뷰 기준과 Severity 실패·복구 시연

재현

변경과 무관한 파일 의견은 게시 후보에서 제외한다.

복구

correctness·security·data
loss·contract break에 집중하고 style
은 linter에 맡긴다.

확인: 변경과 무관한 파일 의견은 게시 후보에서 제외한다.

코드 리뷰 기준과 Severity 소프트웨어 제작

열 파일

src/course_services/contracts.py
data/day3_review_cases/unsafe_pr.diff

작업 범위

코드 리뷰 기준과 Severity 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

한 finding에는 path·line·severity·근거·제안이 모두 있다.
review_rubric.md 저장

코드 리뷰 기준과 Severity Notebook 셀

materials/day3/day3_service_lab.ipynb · 1차시

```
review_rubric = {
    "required_fields": ["path", "line", "severity", "title", "body", "evidence", "suggestion", "confidence",
    "rule_id"],
    "severity": {"P0": "즉시 악용·데이터 손실", "P1": "외부 상태·핵심 기능 위험", "P2": "복구·운영 품질 저하", "P3": "국소적 개선"},
    "excluded": ["style_only", "unrelated_file", "invented_runtime_result"],
}
save_json("01_review_rubric.json", review_rubric)
review_rubric
```

코드 리뷰 기준과 Severity Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k maps_findings
```

저장 위치

```
output/course-labs/day3/01_review_rubric.json
```

한 finding에는 path·line·severity·근거·제안이 모두 있다.
실패 시 review_rubric.md의 status·error_code를 먼저 확인

코드 리뷰 기준과 Severity 실패 증거

변경과 무관한 파일 의견은 게시 후보에서 제외한다.

```
result_file = "output/course-labs/day3/01_review_rubric.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

review_rubric.md이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

코드 리뷰 기준과 Severity 정상 Case

NORMAL

한 finding에는 path·line·severity·근거·제안이 모두 있다.

- 01 — AGENTS.md 입력과 command가 기록됐는가
- 02 — review_rubric.md schema가 validate되는가
- 03 — 코드 리뷰 기준과 Severity focused test가 실제로 통과했는가

코드 리뷰 기준과 Severity 경계 Case

BOUNDARY

변경과 무관한 파일 의견은 게시 후보에서 제외한다.

- 01 — 코드 리뷰 기준과 Severity: raw traceback 대신 stable error_code가 남는가
- 02 — review_rubric.md: 외부 쓰기·자동 메일이 false인가
- 03 — 변경과 무관한 파일 의견은 게시 후보에서 제외한다. 재현이 가능한가

코드 리뷰 기준과 Severity 현업 적용

작은 팀의 PR 대기 시간을 줄이는 첫 번째 자동 검토

- 01 — 코드 리뷰 기준과 Severity은 합성·비식별 sample로 먼저 실행
- 02 — 작은 팀의 PR 대기 시간을 줄이는 첫 번째 자동 검토의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — review_rubric.md 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_rubric.md을 운영 검토 자료로 사용

코드 리뷰 기준과 Severity 포트폴리오 적용

코드 리뷰 기준과 Severity · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다.의 사용자 영향을 한 문장으로 설명
- 02 — 코드 리뷰 기준과 Severity 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — AGENTS.md 리뷰 기준을 읽고 바뀐 라인에만 finding을 작성하게 한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — review_rubric.md과 README 재현 절차를 제출

코드 리뷰 기준과 Severity 운영 점검

품질

무엇을 한 finding에는 path·line·severity·근거·제안이 모두 있다.로 정의하는가

복구

모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다. 발생 시 review_rubric.md에서 원인 상태를 확인

안전

코드 리뷰 기준과 Severity의 사람 승인과 external_write=false 위치

관측

review_rubric.md에서 어떤 status·error_code를 보는가

review_rubric.md 실행 증거

- 01 — 설명: 코드 리뷰 기준과 Severity이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 03 — 증거: review_rubric.md과 정상·실패 test 결과가 남아 있다.

다음 차시: Unified Diff와 Line Mapping

DAY 3 · 2차시

09:50-10:40 (쉬는 시간 11:30-12:00)

Unified Diff와 Line Mapping

Diff · Hunk · parsed_hunks.json

진행

Unified Diff와 Line Mapping
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/
review_service.py
tests/test_course_services.py

확인할 결과

parsed_hunks.json

2차시 입력과 결과

입력



review_rubric.md

이번 차시



Unified Diff와 Line Mapping

결과



parsed_hunks.json

다음 연결



리뷰 Context Pack

Unified Diff와 Line Mapping 필요성

내용이 맞아도 line mapping이 틀리면 GitHub comment는 422 또는 엉뚱한 위치에 붙는다.

+++ target 확인에서 parsed_hunks.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: `parsed_hunks.json`

Unified Diff와 Line Mapping 학습 결과

- 01 — hunk header로 새 파일 line number를 복원한다.
- 02 — 파일 header와 코드의 + 기호를 같은 것으로 처리한다. 왜 위험한지 설명한다.
- 03 — parsed_hunks.json에서 정상과 실패 상태를 직접 확인한다.

Unified Diff와 Line Mapping 용어

용어	이 수업에서 쓰는 뜻
Diff	변경 전후 차이
Hunk	연속된 변경 구간
Old line	변경 전 파일의 줄
New line	변경 후 파일의 줄

parsed_hunks.json 입출력

INPUT

src/course_services/review_service.py

OUTPUT

parsed_hunks.json

PROCESS

+++ target 확인 → @@ header parse → context는 양쪽
증가

VERIFY

추가된 9·11·12번 줄에 finding이 매핑된다.
absolute path와 hunk 없는 입력은 구조화 실패다.

Unified Diff와 Line Mapping 실행 흐름

01

+++ target 확인

02

@@ header parse

03

context는 양쪽 증가

04

삭제는 old만 증가

05

추가는 new에 기록

마지막 판단은 `parsed_hunks.json`에서 사람이 확인합니다.

parsed_hunks.json 정상·실패 계약

정상	——	추가된 9·11·12번 줄에 finding이 매핑된다.
경계	——	absolute path와 hunk 없는 입력은 구조화 실패다.
외부 쓰기	——	Unified Diff와 Line Mapping: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·parsed_hunks.json

Unified Diff와 Line Mapping 파일 지도

입력·fixture

src/course_services/review_service.py

구현 코드

tests/test_course_services.py

검증·test

data/day3_review_cases/unsafe_pr.diff

따라하기 notebook

materials/day3/day3_service_lab.ipynb

Unified Diff와 Line Mapping 개념·코드

Diff

변경 전후 차이

```
src/course_services/review_service.py
```

Hunk

연속된 변경 구간

```
tests/test_course_services.py
```

Old line

변경 전 파일의 줄

```
data/day3_review_cases/unsafe_pr.diff
```

New line

변경 후 파일의 줄

```
materials/day3/day3_service_lab.ipynb
```


Unified Diff와 Line Mapping 선택 기준

구분	역할	이번 차시의 판단 기준
Diff	+++ target 확인 단계의 입력 기준	결정론 test로 먼저 확인
Hunk	@@ header parse 단계의 교체 경계	adapter 경계를 확인
Old line	context는 양쪽 증가 단계의 운영 조건	비용·지연·권한을 확인
New line	삭제는 old만 증가 단계의 판단 기록	사람 판단과 운영 기록을 확인

Unified Diff와 Line Mapping 대표 실패

실패

파일 header와 코드의 + 기호를 같은
것으로 처리한다.

사용자 영향

파일 header와 코드의 + 기호를 같은 것으로
처리한다. 상태에서는 `parsed_hunks.json`을
운영 판단에 사용할 수 없다.

Unified Diff와 Line Mapping 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

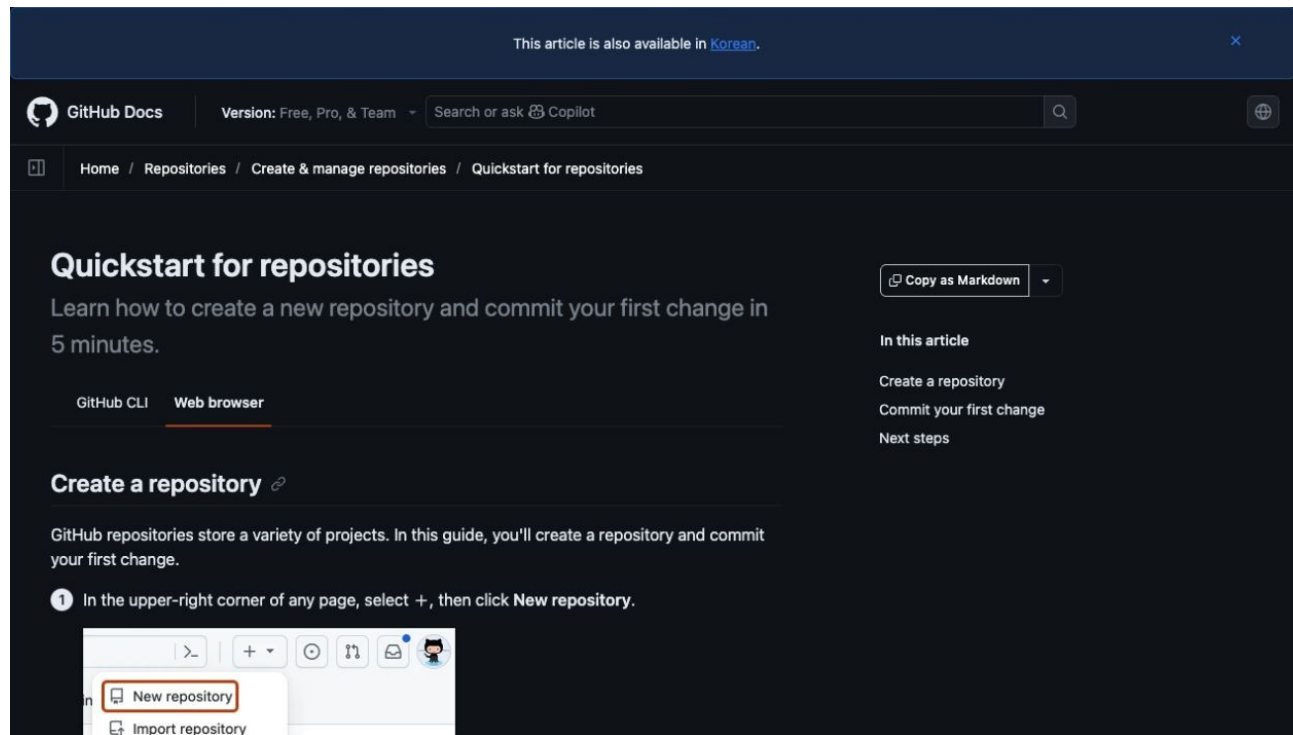
Unified Diff와 Line Mapping 복구 경로

- 01 — 탐지: absolute path와 hunk 없는 입력은 구조화 실패다.
- 02 — 중단: 파일 header와 코드의 + 기호를 같은 것으로 처리한다.
- 03 — 복구: +++ header를 먼저 분리하고 hunk 안의 추가 라인만 수집한다.
- 04 — 증거: parsed_hunks.json과 test 결과를 함께 남긴다.

Unified Diff와 Line Mapping Codex 검증 루프

tests/test_course_services.py
의 책임을 찾고
parsed_hunks.json 계약을
focused test로 고정합니다.

Unified Diff와 Line Mapping
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Unified Diff와 Line Mapping Codex 작업 명세

목표

rename·new file·삭제 only diff를 포함한 parser boundary test를 제안받는다.

허용 경로

tests/test_course_services.py
data/day3_review_cases/unsafe_pr.diff

완료 조건

추가된 9·11·12번 줄에 finding이 매핑된다.
absolute path와 hunk 없는 입력은 구조화 실패다.

금지

Unified Diff와 Line Mapping: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Unified Diff와 Line Mapping 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | Unified Diff와 Line Mapping 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 추가된 9·11·12번 줄에 finding이 매핑된다. |
| 03 | Claude review | —— | parsed_hunks.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Unified Diff와 Line Mapping의 두 LLM 의견과 test는 자동 승인이 아니다 |

Unified Diff와 Line Mapping 정상 시연

- 01 — 열기: `src/course_services/review_service.py`
- 02 — 구현 확인: `tests/test_course_services.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k unified_diff`
- 04 — 성공 신호: 추가된 9·11·12번 줄에 finding이 매핑된다.

Unified Diff와 Line Mapping 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k unified_diff
```

실행 전 확인

Unified Diff와 Line Mapping: repository root · Python 3.12 · .env 미출력

02_parsed_hunks.json 실행 결과

```
{
  "changed_paths": [
    "src/payment_job.py"
  ],
  "added_lines": [
    {
      "path": "src/payment_job.py",
      "line": 9,
      "text": "    result = eval(command)"
    },
    {
      "path": "src/payment_job.py",
      "line": 10,
      "text": "    try:"
    },
    {
      "path": "src/payment_job.py",
      "line": 11,
      "text": "        requests.post(\"https://example.invalid/jobs\", json=payload)"
    },
    {
      "path": "src/payment_job.py",
      "line": 12,
      "text": "    except Exception:"
    },
    ...
  ]
}
```

Unified Diff와 Line Mapping 실패·복구 시연

재현

absolute path와 hunk 없는 입력은 구조화 실패다.

복구

+++ header를 먼저 분리하고 hunk 안의 추가 라인만 수집한다.

확인: absolute path와 hunk 없는 입력은 구조화 실패다.

Unified Diff와 Line Mapping 소프트웨어 제작

열 파일	<code>tests/test_course_services.py</code> <code>data/day3_review_cases/unsafe_pr.diff</code>
작업 범위	Unified Diff와 Line Mapping 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	추가된 9·11·12번 줄에 finding이 매핑된다. <code>parsed_hunks.json</code> 저장

Unified Diff와 Line Mapping Notebook 셀

materials/day3/day3_service_lab.ipynb · 2차시

```
from src.course_services.review_service import parse_unified_diff

diff_path = ROOT / "data/day3_review_cases/unsafe_pr.diff"
diff_text = diff_path.read_text(encoding="utf-8")
parsed = parse_unified_diff(diff_text)
parsed_result = {
    "changed_paths": parsed.changed_paths,
    "added_lines": [{"path": line.path, "line": line.line, "text": line.text} for line in parsed.added_lines],
}
save_json("02_parsed_hunks.json", parsed_result)
[(line.line, line.text) for line in parsed.added_lines]
```

Unified Diff와 Line Mapping Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k unified_diff
```

저장 위치

```
output/course-labs/day3/02_parsed_hunks.json
```

추가된 9·11·12번 줄에 finding이 매핑된다.
실패 시 parsed_hunks.json의 status·error_code를 먼저 확인

Unified Diff와 Line Mapping 실패 증거

absolute path와 hunk 없는 입력은 구조화 실패다.

```
result_file = "output/course-labs/day3/02_parsed_hunks.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

parsed_hunks.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Unified Diff와 Line Mapping 정상 Case

NORMAL

추가된 9·11·12번 줄에 finding이 매핑된다.

- 01 — src/course_services/review_service.py 입력과 command가 기록됐는가
- 02 — parsed_hunks.json schema가 validate되는가
- 03 — Unified Diff와 Line Mapping focused test가 실제로 통과했는가

Unified Diff와 Line Mapping 경계 Case

BOUNDARY

absolute path와 hunk 없는 입력은 구조화 실패다.

- 01 — Unified Diff와 Line Mapping: raw traceback 대신 stable error_code가 남는가
- 02 — parsed_hunks.json: 외부 쓰기·자동 메일이 false인가
- 03 — absolute path와 hunk 없는 입력은 구조화 실패다. 재현이 가능한가

Unified Diff와 Line Mapping 현업 적용

GitHub·GitLab에서 공통으로 쓰는 diff 분석 core

- 01 — Unified Diff와 Line Mapping은 합성·비식별 sample로 먼저 실행
- 02 — GitHub·GitLab에서 공통으로 쓰는 diff 분석 core의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — parsed_hunks.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — parsed_hunks.json을 운영 검토 자료로 사용

Unified Diff와 Line Mapping 포트폴리오 적용

Unified Diff와 Line Mapping · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 파일 header와 코드의 + 기호를 같은 것으로 처리한다.의 사용자 영향을 한 문장으로 설명
- 02 — Unified Diff와 Line Mapping 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — rename·new file·삭제 only diff를 포함한 parser boundary test를 제안받는다.의 diff와 직접 검토한 test 증거를 구분
- 04 — parsed_hunks.json과 README 재현 절차를 제출

Unified Diff와 Line Mapping 운영 점검

품질

무엇을 추가된 9·11·12번 줄에 finding이 매핑된다.로 정의하는가

복구

파일 header와 코드의 + 기호를 같은 것으로 처리한다.
발생 시 parsed_hunks.json에서 원인 상태를 확인

안전

Unified Diff와 Line Mapping의 사람 승인과
external_write=false 위치

관측

parsed_hunks.json에서 어떤 status·error_code를
보는가

parsed_hunks.json 실행 증거

- 01 — 설명: Unified Diff와 Line Mapping이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k unified_diff`
- 03 — 증거: `parsed_hunks.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 리뷰 Context Pack

DAY 3 · 3차시

10:40-11:30 (쉬는 시간 11:30-12:00)

리뷰 Context Pack

Repo context · Dependency · context_pack.json

진행

리뷰 Context Pack
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

AGENTS.md
src/course_services/
review_service.py

확인할 결과

context_pack.json

3차시 입력과 결과

입력	————	parsed_hunks.json
이번 차시	————	리뷰 Context Pack
결과	————	context_pack.json
다음 연결	————	ReviewFinding 결과 계약

리뷰 Context Pack 필요성

파일을 많이 넣으면 정확도가 자동으로 오르는 대신 비용과 상충 정보가 늘어난다.

changed path에서 context_pack.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: context_pack.json

리뷰 Context Pack 학습 결과

- 01 — 변경 파일·직접 호출부·관련 test만 context pack으로 고른다.
- 02 — 비밀키·대용량 로그·무관한 output까지 context에 넣는다. 왜 위험한지 설명한다.
- 03 — context_pack.json에서 정상과 실패 상태를 직접 확인한다.

리뷰 Context Pack 용어

용어	이 수업에서 쓰는 뜻
Repo context	변경을 이해하는 데 필요한 저장소 정보
Dependency	변경 코드가 의존하는 대상
Call site	함수를 실제 호출하는 위치
Context budget	한 번에 전달할 문맥 한도

context_pack.json 입출력

INPUT

AGENTS.md

OUTPUT

context_pack.json

PROCESS

changed path → 관련 symbol → 가까운 test

VERIFY

context pack에 changed_paths와 test_candidates가 남는다.
workspace 밖·.env·실행 cache는 context에서 제외한다.

리뷰 Context Pack 실행 흐름

01

changed path

02

관련 symbol

03

가까운 test

04

정책 문서

05

불필요 문맥 제외

마지막 판단은 context_pack.json에서 사람이 확인합니다.

context_pack.json 정상·실패 계약

정상	—— context pack에 changed_paths와 test_candidates가 남는다.
경계	—— workspace 밖·.env·실행 cache는 context에서 제외한다.
외부 쓰기	—— 리뷰 Context Pack: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·context_pack.json

리뷰 Context Pack 파일 지도

입력·fixture

AGENTS.md

구현 코드

src/course_services/review_service.py

검증·test

tests/test_course_services.py

따라하기 notebook

README.md

리뷰 Context Pack 개념·코드

Repo context

변경을 이해하는 데 필요한 저장소 정보

```
AGENTS.md
```

Dependency

변경 코드가 의존하는 대상

```
src/course_services/review_service.py
```

Call site

함수를 실제 호출하는 위치

```
tests/test_course_services.py
```

Context budget

한 번에 전달할 문맥 한도

```
README.md
```

리뷰 Context Pack 선택 기준

구분	역할	이번 차시의 판단 기준
Repo context	changed path 단계의 입력 기준	결정론 test로 먼저 확인
Dependency	관련 symbol 단계의 교체 경계	adapter 경계를 확인
Call site	가까운 test 단계의 운영 조건	비용·지연·권한을 확인
Context budget	정책 문서 단계의 판단 기록	사람 판단과 운영 기록을 확인

리뷰 Context Pack 대표 실패

실패

비밀키·대용량 로그·무관한 output까지 context에 넣는다.

사용자 영향

비밀키·대용량 로그·무관한 output까지 context에 넣는다. 상태에서는 context_pack.json을 운영 판단에 사용할 수 없다.

리뷰 Context Pack 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

리뷰 Context Pack 복구 경로

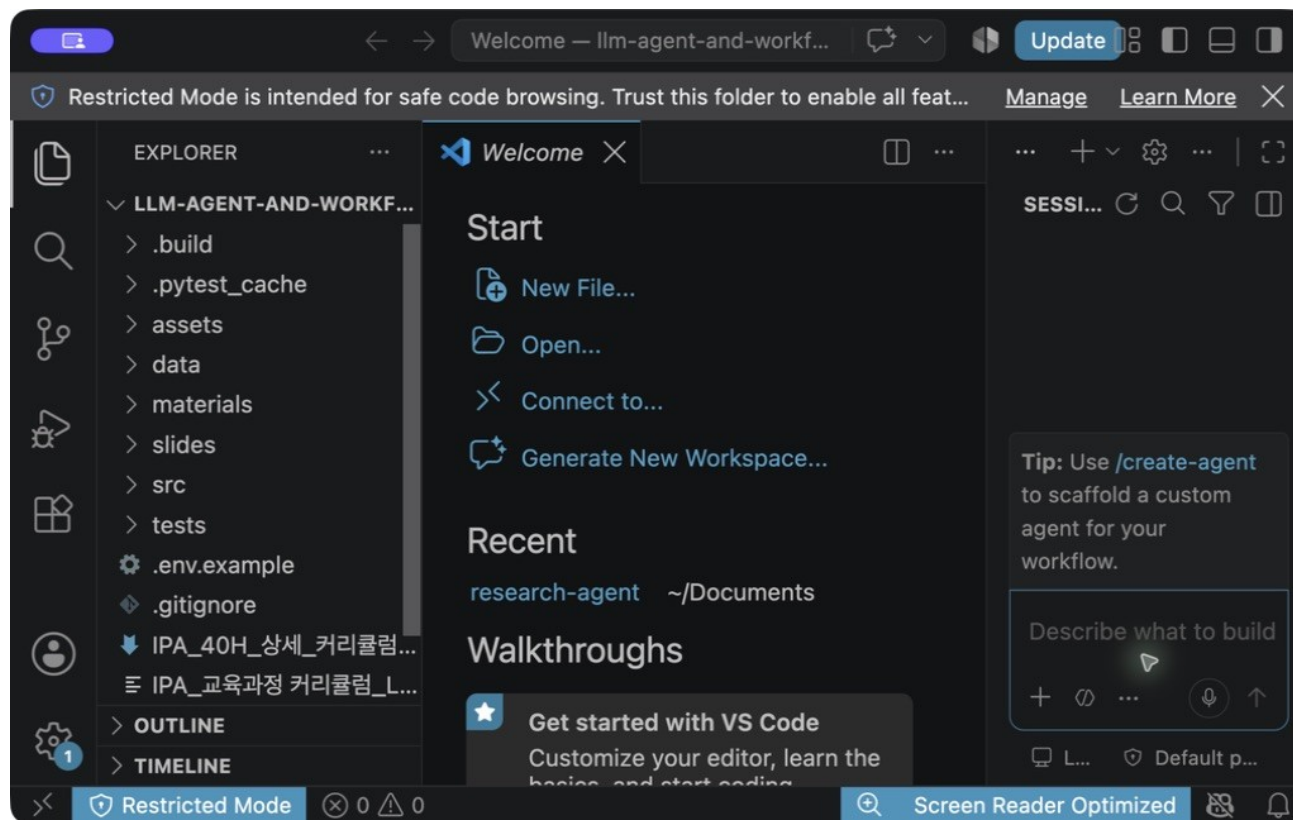
- 01 — 탐지: workspace 밖·.env·실행 cache는 context에서 제외한다.
- 02 — 중단: 비밀키·대용량 로그·무관한 output까지 context에 넣는다.
- 03 — 복구: rg로 후보를 좁히고 secret·generated output을 명시적으로 제외한다.
- 04 — 증거: context_pack.json과 test 결과를 함께 남긴다.

리뷰 Context Pack Codex 검증 루프

`src/course_services/
review_service.py`의 책임을 찾고
`context_pack.json` 계약을
focused test로 고정합니다.

리뷰 Context Pack

저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



리뷰 Context Pack Codex 작업 명세

목표

저장소를 이해하되 수정 전 관련 파일과 test를 먼저 나열하게 한다.

허용 경로

src/course_services/review_service.py
tests/test_course_services.py

완료 조건

context pack에 changed_paths와 test_candidates가 남는다.
workspace 밖 .env 실행 cache는 context에서 제외한다.

금지

리뷰 Context Pack: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

리뷰 Context Pack 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | 리뷰 Context Pack 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | context pack에 changed_paths와 test_candidates가 남는다. |
| 03 | Claude review | —— | context_pack.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 리뷰 Context Pack의 두 LLM 의견과 test는 자동 승인이 아니다 |

리뷰 Context Pack 정상 시연

- 01 — 열기: AGENTS.md
- 02 — 구현 확인: src/course_services/review_service.py
- 03 — 실행: rg -n "run_review_service|ReviewFinding" src tests
- 04 — 성공 신호: context pack에 changed_paths와 test_candidates가 남는다.

리뷰 Context Pack 실행 명령

```
$ rg -n "run_review_service|ReviewFinding" src tests
```

실행 전 확인

리뷰 Context Pack: repository root · Python 3.12 · .env 미출력

03_context_pack.json 실행 결과

```
{
  "changed_paths": [
    "src/payment_job.py"
  ],
  "added_line_count": 6,
  "test_candidates": [
    "tests/test_payment_job.py"
  ],
  "review_focus": [
    "correctness",
    "security",
    "data_loss",
    "contract_break"
  ],
  "excluded_focus": [
    "style_only",
    "unrelated_files",
    "invented_runtime_result"
  ]
}
```

리뷰 Context Pack 실패·복구 시연

재현

workspace 밖·.env·실행 cache는 context에서 제외한다.

복구

rg로 후보를 좁히고 secret·generated output을 명시적으로 제외한다.

확인: workspace 밖·.env·실행 cache는 context에서 제외한다.

리뷰 Context Pack 소프트웨어 제작

열 파일

`src/course_services/review_service.py`
`tests/test_course_services.py`

작업 범위

리뷰 Context Pack 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

context pack에 `changed_paths`와 `test_candidates`가 남는다.
`context_pack.json` 저장

리뷰 Context Pack Notebook 셀

materials/day3/day3_service_lab.ipynb · 3차시

```
from src.course_services.review_service import build_context_pack

context_pack = build_context_pack(parsed)
assert "style_only" in context_pack["excluded_focus"]
save_json("03_context_pack.json", context_pack)
context_pack
```

리뷰 Context Pack Terminal 실행

```
$ rg -n "run_review_service|ReviewFinding" src tests
```

저장 위치

```
output/course-labs/day3/03_context_pack.json
```

context pack에 changed_paths와 test_candidates가 남는다.
실패 시 context_pack.json의 status·error_code를 먼저 확인

리뷰 Context Pack 실패 증거

workspace 밖·.env·실행 cache는 context에서 제외한다.

```
result_file = "output/course-labs/day3/03_context_pack.json"
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"
external_write = false
```

context_pack.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

리뷰 Context Pack 정상 Case

NORMAL

context pack에 changed_paths와 test_candidates가 남는다.

- 01 — AGENTS.md 입력과 command가 기록됐는가
- 02 — context_pack.json schema가 validate되는가
- 03 — 리뷰 Context Pack focused test가 실제로 통과했는가

리뷰 Context Pack 경계 Case

BOUNDARY

workspace 밖·.env·실행 cache는 context에서 제외한다.

- 01 — 리뷰 Context Pack: raw traceback 대신 stable error_code가 남는가
- 02 — context_pack.json: 외부 쓰기·자동 메일이 false인가
- 03 — workspace 밖·.env·실행 cache는 context에서 제외한다. 재현이 가능한가

리뷰 Context Pack 현업 적용

수천 파일 monorepo에서 리뷰 latency와 비용을 줄이는 selection

- 01 — 리뷰 Context Pack은 합성·비식별 sample로 먼저 실행
- 02 — 수천 파일 monorepo에서 리뷰 latency와 비용을 줄이는 selection의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — context_pack.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — context_pack.json을 운영 검토 자료로 사용

리뷰 Context Pack 포트폴리오 적용

리뷰 Context Pack · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 비밀키·대용량 로그·무관한 output까지 context에 넣는다.의 사용자 영향을 한 문장으로 설명
- 02 — 리뷰 Context Pack 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 저장소를 이해하되 수정 전 관련 파일과 test를 먼저 나열하게 한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — context_pack.json과 README 재현 절차를 제출

리뷰 Context Pack 운영 점검

품질

무엇을 context pack에 changed_paths와 test_candidates가 남는다.로 정의하는가

복구

비밀키·대용량 로그·무관한 output까지 context에 넣는다. 발생 시 context_pack.json에서 원인 상태를 확인

안전

리뷰 Context Pack의 사람 승인과 external_write=false 위치

관측

context_pack.json에서 어떤 status·error_code를 보는가

context_pack.json 실행 증거

- 01 — 설명: 리뷰 Context Pack이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `rg -n "run_review_service|ReviewFinding" src tests`
- 03 — 증거: context_pack.json과 정상·실패 test 결과가 남아 있다.

다음 차시: ReviewFinding 결과 계약

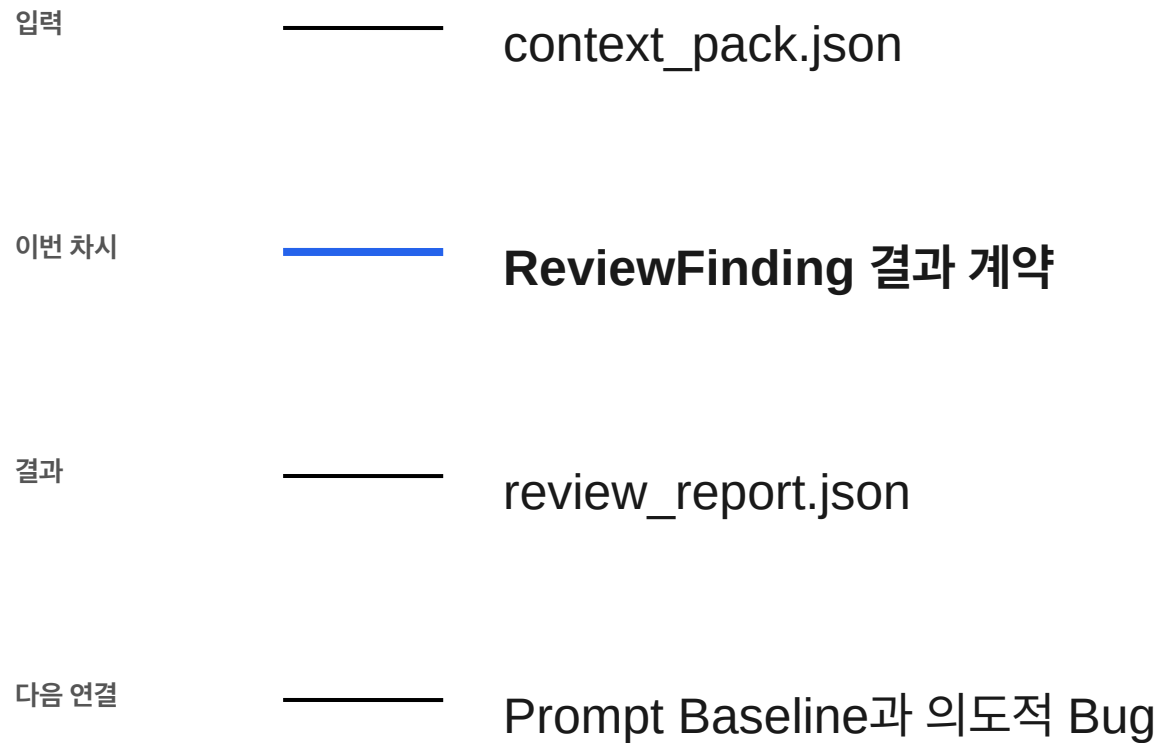
12:00-12:50 (쉬는 시간 13:40-14:00)

ReviewFinding 결과 계약

Contract · extra=forbid · review_report.json

진행	사용 파일	확인할 결과
ReviewFinding 결과 계약 강의 12분 · 강사 시연 10분 · Codex 소프트웨어 제작 23분 · 실행 확인 5분	src/course_services/contracts.py src/course_services/ review_service.py	review_report.json

4차시 입력과 결과



ReviewFinding 결과 계약 필요성

자유 문장은 deduplication·line comment·평가에 다시 쓰기 어렵다.

finding validate에서 review_report.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: review_report.json

ReviewFinding 결과 계약 학습 결과

- 01 — ReviewFinding과 ReviewReport를 Pydantic으로 검증한다.
- 02 — schema 오류가 raw traceback으로 notebook을 끝낸다. 왜 위험한지 설명한다.
- 03 — review_report.json에서 정상과 실패 상태를 직접 확인한다.

ReviewFinding 결과 계약 용어

용어	이 수업에서 쓰는 뜻
Contract	호출자와 구현 사이의 약속
extra=forbid	모르는 필드를 거부
Error code	복구 판단에 쓰는 이름
Automatic publish	사람 없이 게시되는 동작

review_report.json 입출력

INPUT

src/course_services/contracts.py

OUTPUT

review_report.json

PROCESS

finding validate → line mapping → report aggregate

VERIFY

confidence 0~1과 line $\geq$ 1이 검증된다.
automatic_publish=true는 model validation에서 차단된다.

ReviewFinding 결과 계약 실행 흐름

01

finding validate

02

line mapping

03

report aggregate

04

automatic_publish
false

05

expected failure
code

마지막 판단은 review_report.json에서 사람이 확인합니다.

review_report.json 정상·실패 계약

정상	——	confidence 0~1과 line $\geq$ 10이 검증된다.
경계	——	automatic_publish=true는 model validation에서 차단된다.
외부 쓰기	——	ReviewFinding 결과 계약: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·review_report.json

ReviewFinding 결과 계약 파일 지도

입력·fixture

`src/course_services/contracts.py`

구현 코드

`src/course_services/review_service.py`

검증·test

`tests/test_course_services.py`

따라하기 notebook

`data/day5_eval/golden_review_findings.json`

ReviewFinding 결과 계약 개념·코드

Contract

호출자와 구현 사이의 약속

```
src/course_services/contracts.py
```

extra=forbid

모르는 필드를 거부

```
src/course_services/review_service.py
```

Error code

복구 판단에 쓰는 이름

```
tests/test_course_services.py
```

Automatic publish

사람 없이 게시되는 동작

```
data/day5_eval/golden_review_findings.json
```

ReviewFinding 결과 계약 선택 기준

구분	역할	이번 차시의 판단 기준
Contract	finding validate 단계의 입력 기준	결정론 test로 먼저 확인
extra=forbid	line mapping 단계의 교체 경계	adapter 경계를 확인
Error code	report aggregate 단계의 운영 조건	비용·지연·권한을 확인
Automatic publish	automatic_publish false 단계의 판단 기록	사람 판단과 운영 기록을 확인

ReviewFinding 결과 계약 대표 실패

실패

schema 오류가 raw traceback으로 notebook을 끝낸다.

사용자 영향

schema 오류가 raw traceback으로 notebook을 끝낸다. 상태에서는 review_report.json을 운영 판단에 사용할 수 없다.

ReviewFinding 결과 계약 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

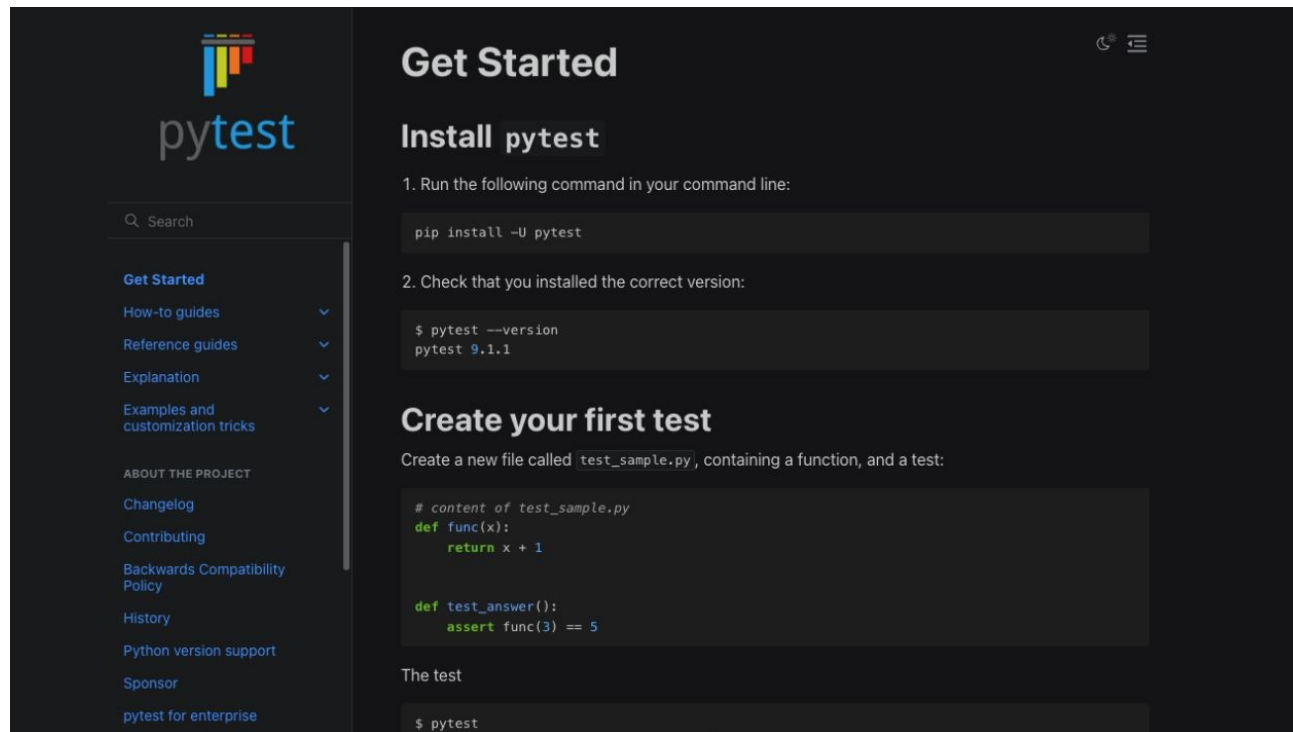
ReviewFinding 결과 계약 복구 경로

- 01 — 탐지: `automatic_publish=true`는 `model validation`에서 차단된다.
- 02 — 중단: `schema` 오류가 `raw traceback`으로 `notebook`을 끝낸다.
- 03 — 복구: `EXPECTED_FAILURE`와 `error_code`로 바꾸고 원인은 `trace`에 남긴다.
- 04 — 증거: `review_report.json`과 `test` 결과를 함께 남긴다.

ReviewFinding 결과 계약 Codex 검증 루프

src/course_services/
review_service.py의 책임을 찾고
review_report.json 계약을
focused test로 고정합니다.

ReviewFinding 결과 계약
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



ReviewFinding 결과 계약 Codex 작업 명세

목표

contract 변경 시 정상·누락 field·extra field test를 함께 갱신한다.

허용 경로

src/course_services/review_service.py
tests/test_course_services.py

완료 조건

confidence 0~1과 line $\geq$ 1이 검증된다.
automatic_publish=true는 model validation에서 차단된다.

금지

ReviewFinding 결과 계약: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

ReviewFinding 결과 계약 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | ReviewFinding 결과 계약 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | confidence 0~1과 line>=1이 검증된다. |
| 03 | Claude review | —— | review_report.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | ReviewFinding 결과 계약의 두 LLM 의견과 test는 자동 승인이 아니다 |

ReviewFinding 결과 계약 정상 시연

- 01 — 열기: `src/course_services/contracts.py`
- 02 — 구현 확인: `src/course_services/review_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: confidence 0~1과 line $\geq$ 1이 검증된다.

ReviewFinding 결과 계약 실행 명령

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

ReviewFinding 결과 계약: repository root · Python 3.12 · .env 미출력

04_review_contract.json 실행 결과

```
{
  "normal": {
    "status": "SUCCESS",
    "findings": [
      {
        "path": "src/payment_job.py",
        "line": 9,
        "severity": "P0",
        "title": "검증되지 않은 문자열이 코드로 실행될 수 있음",
        "body": "입력 경로에 따라 임의 코드 실행으로 이어질 수 있습니다.",
        "evidence": "result = eval(command)",
        "suggestion": "허용된 명령이나 파서만 호출하는 명시적 registry로 교체하세요.",
        "confidence": 0.99,
        "rule_id": "unsafe-dynamic-execution"
      },
      {
        "path": "src/payment_job.py",
        "line": 11,
        "severity": "P1",
        "title": "외부 쓰기 전에 사람 승인 경계가 보이지 않음",
        "body": "재시도나 잘못된 대상 선택이 실제 게시·수정으로 이어질 수 있습니다.",
        "evidence": "requests.post(\"https://example.invalid/jobs\", json=payload)",
        "suggestion": "dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.",
        "confidence": 0.91,
        "rule_id": "external-write-without-approval"
      },
      ...
    ]
  }
}
```

ReviewFinding 결과 계약 실패·복구 시연

재현

automatic_publish=true는 model validation에서 차단된다.

복구

EXPECTED_FAILURE와 error_code로 바꾸고 원인은 trace에 남긴다.

확인: automatic_publish=true는 model validation에서 차단된다.

ReviewFinding 결과 계약 소프트웨어 제작

열 파일 `src/course_services/review_service.py`
 `tests/test_course_services.py`

작업 범위 **ReviewFinding 결과 계약 명세 → 최소 Patch → Focused Test → Diff 검토**

완료 기준 **confidence 0~1과 line>=10이 검증된다.**
 review_report.json 저장

ReviewFinding 결과 계약 Notebook 셀

materials/day3/day3_service_lab.ipynb · 4차시

```
from pydantic import ValidationError
from src.course_services.contracts import ReviewReport
from src.course_services.review_service import run_review_service

review_report = run_review_service(diff_text)
validated_report = ReviewReport.model_validate(review_report)
try:
    ReviewReport(status="SUCCESS", automatic_publish=True)
    contract_boundary = {"status": "UNEXPECTED_SUCCESS"}
except ValidationError as exc:
    contract_boundary = {"status": "EXPECTED_FAILURE", "error_code": str(exc).splitlines()[0]}
save_json("04_review_contract.json", {"normal": validated_report.model_dump(mode="json"), "boundary":
contract_boundary})
{"finding_count": len(validated_report.findings), "boundary": contract_boundary}
```


ReviewFinding 결과 계약 Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py
```

저장 위치

```
output/course-labs/day3/04_review_contract.json
```

confidence 0~1과 line>=1이 검증된다.

실패 시 review_report.json의 status·error_code를 먼저 확인

ReviewFinding 결과 계약 실패 증거

`automatic_publish=true`는 model validation에서 차단된다.

```
result_file = "output/course-labs/day3/04_review_contract.json"
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"
external_write = false
```

`review_report.json`이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

ReviewFinding 결과 계약 정상 Case

NORMAL

confidence 0~1과 line>=1이 검증된다.

- 01 — src/course_services/contracts.py 입력과 command가 기록됐는가
- 02 — review_report.json schema가 validate되는가
- 03 — ReviewFinding 결과 계약 focused test가 실제로 통과했는가

ReviewFinding 결과 계약 경계 Case

BOUNDARY

automatic_publish=true는 model validation에서 차단된다.

- 01 — ReviewFinding 결과 계약: raw traceback 대신 stable error_code가 남는가
- 02 — review_report.json: 외부 쓰기·자동 메일이 false인가
- 03 — automatic_publish=true는 model validation에서 차단된다. 재현이 가능한가

ReviewFinding 결과 계약 현업 적용

리뷰 결과를 IDE·PR·대시보드에서 같은 형식으로 재사용

- 01 — ReviewFinding 결과 계약은 합성·비식별 sample로 먼저 실행
- 02 — 리뷰 결과를 IDE·PR·대시보드에서 같은 형식으로 재사용의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — review_report.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_report.json을 운영 검토 자료로 사용

ReviewFinding 결과 계약 포트폴리오 적용

ReviewFinding 결과 계약 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — schema 오류가 raw traceback으로 notebook을 끝낸다.의 사용자 영향을 한 문장으로 설명
- 02 — ReviewFinding 결과 계약 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — contract 변경 시 정상·누락 field·extra field test를 함께 갱신한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — review_report.json과 README 재현 절차를 제출

ReviewFinding 결과 계약 운영 점검

품질

무엇을 confidence 0~1과 line>=1이 검증된다.로 정의하는가

복구

schema 오류가 raw traceback으로 notebook을 끝낸다. 발생 시 review_report.json에서 원인 상태를 확인

안전

ReviewFinding 결과 계약의 사람 승인과 external_write=false 위치

관측

review_report.json에서 어떤 status·error_code를 보는가

review_report.json 실행 증거

- 01 — 설명: ReviewFinding 결과 계약이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: review_report.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Prompt Baseline과 의도적 Bug

DAY 3 · 5차시

12:50-13:40 (쉬는 시간 13:40-14:00)

Prompt Baseline과 의도적 Bug

Baseline · Few-shot · baseline_review.json

진행

Prompt Baseline과 의도적 Bug
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

data/day3_review_cases/unsafe_pr.diff
src/course_services/review_service.py

확인할 결과

baseline_review.json

5차시 입력과 결과

입력	————	review_report.json
이번 차시	————	Prompt Baseline과 의도적 Bug
결과	————	baseline_review.json
다음 연결	————	정적 분석·LLM Hybrid Review

Prompt Baseline과 의도적 Bug 필요성

실제 PR만으로 튜닝하면 무엇이 개선됐는지 비교할 정답이 없다.

synthetic diff에서 baseline_review.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: **baseline_review.json**

Prompt Baseline과 의도적 Bug 학습 결과

- 01 — 의도적 bug diff에서 baseline finding을 만든다.
- 02 — LLM이 존재하지 않는 runtime 결과를 근거로 finding을 만든다. 왜 위험한지 설명한다.
- 03 — baseline_review.json에서 정상과 실패 상태를 직접 확인한다.

Prompt Baseline과 의도적 Bug 용어

용어	이 수업에서 쓰는 뜻
Baseline	개선 전 비교 기준
Few-shot	입출력 예시를 prompt에 제공
Rule ID	같은 오류 유형을 묶는 식별자
Deterministic	같은 입력에 재현 가능한 결과

baseline_review.json 입출력

INPUT

data/day3_review_cases/unsafe_pr.diff

OUTPUT

baseline_review.json

PROCESS

synthetic diff → rule baseline → LLM candidate

VERIFY

eval·외부 쓰기·broad except 세 finding이 나온다.
빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

Prompt Baseline과 의도적 Bug 실행 흐름

01

synthetic diff

02

rule baseline

03

LLM candidate

04

schema validate

05

사람 비교

마지막 판단은 `baseline_review.json`에서 사람이 확인합니다.

baseline_review.json 정상·실패 계약

정상	————	eval·외부 쓰기·broad except 세 finding이 나온다.
경계	————	빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
외부 쓰기	————	Prompt Baseline과 의도적 Bug: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	————	status·error_code·입력 식별자·baseline_review.json

Prompt Baseline과 의도적 Bug 파일 지도

입력·fixture

data/day3_review_cases/unsafe_pr.diff

구현 코드

src/course_services/review_service.py

검증·test

materials/day3/day3_service_lab.ipynb

따라하기 notebook

output/day3-review/baseline.json

Prompt Baseline과 의도적 Bug 개념·코드

Baseline

개선 전 비교 기준

```
data/day3_review_cases/unsafe_pr.diff
```

Few-shot

입출력 예시를 prompt에 제공

```
src/course_services/review_service.py
```

Rule ID

같은 오류 유형을 묶는 식별자

```
materials/day3/day3_service_lab.ipynb
```

Deterministic

같은 입력에 재현 가능한 결과

```
output/day3-review/baseline.json
```

Prompt Baseline과 의도적 Bug 선택 기준

구분	역할	이번 차시의 판단 기준
Baseline	synthetic diff 단계의 입력 기준	결정론 test로 먼저 확인
Few-shot	rule baseline 단계의 교체 경계	adapter 경계를 확인
Rule ID	LLM candidate 단계의 운영 조건	비용·지연·권한을 확인
Deterministic	schema validate 단계의 판단 기록	사람 판단과 운영 기록을 확인

Prompt Baseline과 의도적 Bug 대표 실패

실패

LLM이 존재하지 않는 runtime 결과를
근거로 finding을 만든다.

사용자 영향

LLM이 존재하지 않는 runtime 결과를 근거로
finding을 만든다. 상태에서는
baseline_review.json을 운영 판단에 사용할 수
없다.

Prompt Baseline과 의도적 Bug 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

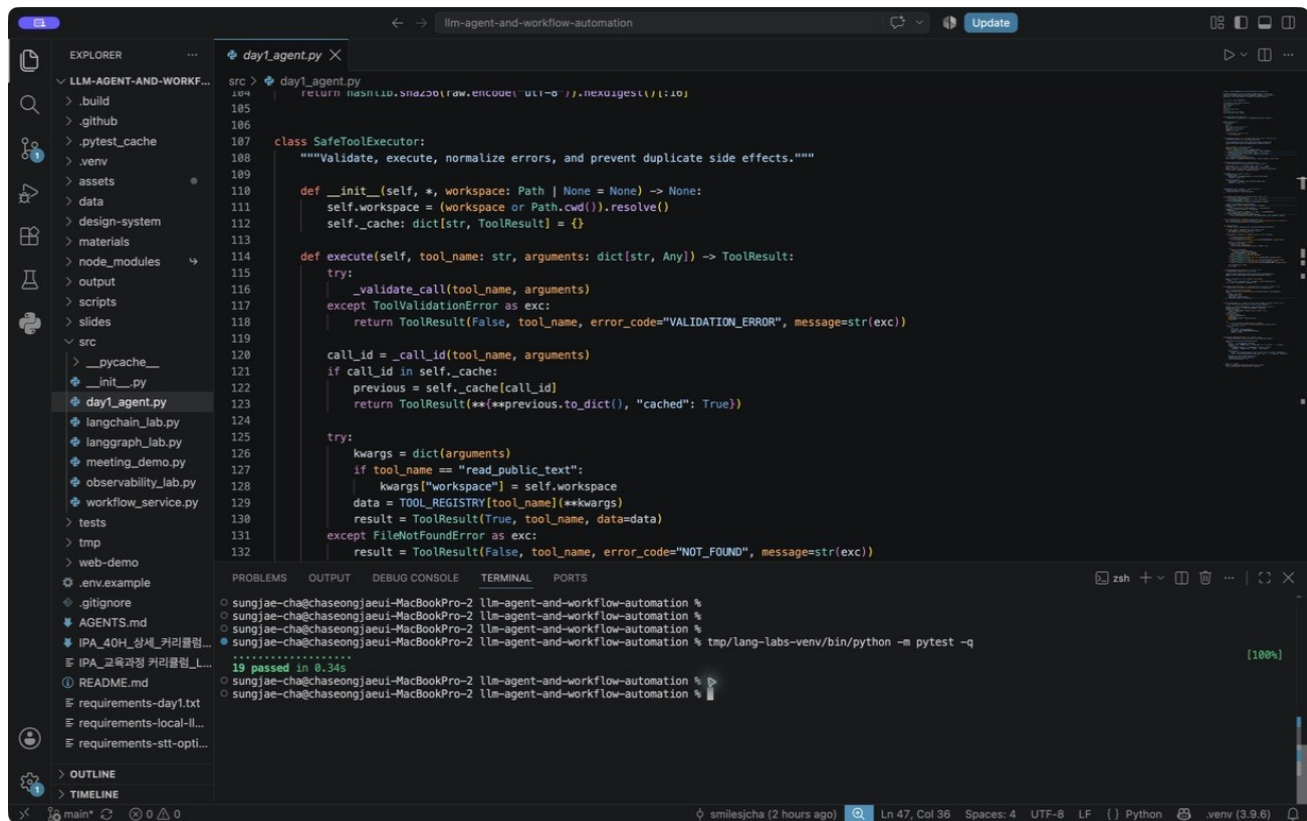
Prompt Baseline과 의도적 Bug 복구 경로

- 01 — 탐지: 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
- 02 — 중단: LLM이 존재하지 않는 runtime 결과를 근거로 finding을 만든다.
- 03 — 복구: diff·static check·실제 test output만 evidence로 허용한다.
- 04 — 증거: baseline_review.json과 test 결과를 함께 남긴다.

Prompt Baseline과 의도적 Bug Codex 검증 루프

src/course_services/
review_service.py의 책임을 찾고
baseline_review.json 계약을
focused test로 고정합니다.

Prompt Baseline과 의도적 Bug
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Prompt Baseline과 의도적 Bug Codex 작업 명세

목표

baseline rule 하나를 구현하고 test가 먼저 실패하는지 확인한 뒤 수정한다.

허용 경로

src/course_services/review_service.py
materials/day3/day3_service_lab.ipynb

완료 조건

eval·외부 쓰기·broad except 세 finding이 나온다.
빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

금지

Prompt Baseline과 의도적 Bug: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Prompt Baseline과 의도적 Bug 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | Prompt Baseline과 의도적 Bug 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | eval·외부 쓰기·broad except 세 finding이 나온다. |
| 03 | Claude review | —— | baseline_review.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Prompt Baseline과 의도적 Bug의 두 LLM 의견과 test는 자동 승인이 아니다 |

Prompt Baseline과 의도적 Bug 정상 시연

- 01 — 열기: `data/day3_review_cases/unsafe_pr.diff`
- 02 — 구현 확인: `src/course_services/review_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 04 — 성공 신호: `eval`·외부 쓰기·`broad except` 세 finding이 나온다.

Prompt Baseline과 의도적 Bug 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k maps_findings
```

실행 전 확인

Prompt Baseline과 의도적 Bug: repository root · Python 3.12 · .env 미출력

05_baseline_review.json 실행 결과

```
{
  "normal": {
    "status": "SUCCESS",
    "findings": [
      {
        "path": "src/payment_job.py",
        "line": 9,
        "severity": "P0",
        "title": "검증되지 않은 문자열이 코드로 실행될 수 있음",
        "body": "입력 경로에 따라 임의 코드 실행으로 이어질 수 있습니다.",
        "evidence": "result = eval(command)",
        "suggestion": "허용된 명령이나 파서만 호출하는 명시적 registry로 교체하세요.",
        "confidence": 0.99,
        "rule_id": "unsafe-dynamic-execution"
      },
      {
        "path": "src/payment_job.py",
        "line": 11,
        "severity": "P1",
        "title": "외부 쓰기 전에 사람 승인 경계가 보이지 않음",
        "body": "재시도나 잘못된 대상 선택이 실제 게시·수정으로 이어질 수 있습니다.",
        "evidence": "requests.post(\"https://example.invalid/jobs\", json=payload)",
        "suggestion": "dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.",
        "confidence": 0.91,
        "rule_id": "external-write-without-approval"
      },
      ...
    ]
  }
}
```

Prompt Baseline과 의도적 Bug 실패·복구 시연

재현

빈 diff는 그럴듯한 빈 성공이 아니라
EMPTY_DIFF다.

복구

diff·static check·실제 test output만
evidence로 허용한다.

확인: 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

Prompt Baseline과 의도적 Bug 소프트웨어 제작

열 파일

`src/course_services/review_service.py`
`materials/day3/day3_service_lab.ipynb`

작업 범위

Prompt Baseline과 의도적 Bug 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

eval·외부 쓰기·broad except 세 finding이 나온다.
baseline_review.json 저장

Prompt Baseline과 의도적 Bug Notebook 셀

materials/day3/day3_service_lab.ipynb · 5차시

```
baseline = run_review_service(diff_text)
rule_ids = [finding["rule_id"] for finding in baseline["findings"]]
empty_diff = run_review_service("")
assert len(rule_ids) == 3
assert empty_diff["error_code"] == "EMPTY_DIFF"
save_json("05_baseline_review.json", {"normal": baseline, "boundary": empty_diff})
rule_ids
```

Prompt Baseline과 의도적 Bug Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k maps_findings
```

저장 위치

```
output/course-labs/day3/05_baseline_review.json
```

eval·외부 쓰기·broad except 세 finding이 나온다.
실패 시 baseline_review.json의 status·error_code를 먼저 확인

Prompt Baseline과 의도적 Bug 실패 증거

빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

```
result_file = "output/course-labs/day3/05_baseline_review.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

baseline_review.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Prompt Baseline과 의도적 Bug 정상 Case

NORMAL

eval·외부 쓰기·broad except 세 finding이 나온다.

- 01 — data/day3_review_cases/unsafe_pr.diff 입력과 command가 기록됐는가
- 02 — baseline_review.json schema가 validate되는가
- 03 — Prompt Baseline과 의도적 Bug focused test가 실제로 통과했는가

Prompt Baseline과 의도적 Bug 경계 Case

BOUNDARY

빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

- 01 — Prompt Baseline과 의도적 Bug: raw traceback 대신 stable error_code가 남는가
- 02 — baseline_review.json: 외부 쓰기·자동 메일이 false인가
- 03 — 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다. 재현이 가능한가

Prompt Baseline과 의도적 Bug 현업 적용

교육용 bug repo에서 reviewer의 품질 변화 설명

- 01 — Prompt Baseline과 의도적 Bug은 합성·비식별 sample로 먼저 실행
- 02 — 교육용 bug repo에서 reviewer의 품질 변화 설명의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — baseline_review.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — baseline_review.json을 운영 검토 자료로 사용

Prompt Baseline과 의도적 Bug 포트폴리오 적용

Prompt Baseline과 의도적 Bug · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — LLM이 존재하지 않는 runtime 결과를 근거로 finding을 만든다.의 사용자 영향을 한 문장으로 설명
- 02 — Prompt Baseline과 의도적 Bug 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — baseline rule 하나를 구현하고 test가 먼저 실패하는지 확인한 뒤 수정한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — baseline_review.json과 README 재현 절차를 제출

Prompt Baseline과 의도적 Bug 운영 점검

품질

무엇을 eval·외부 쓰기·broad except 세 finding이 나온다.로 정의하는가

복구

LLM이 존재하지 않는 runtime 결과를 근거로 finding을 만든다. 발생 시 baseline_review.json에서 원인 상태를 확인

안전

Prompt Baseline과 의도적 Bug의 사람 승인과 external_write=false 위치

관측

baseline_review.json에서 어떤 status·error_code를 보는가

baseline_review.json 실행 증거

- 01 — 설명: Prompt Baseline과 의도적 Bug이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 03 — 증거: baseline_review.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 정적 분석·LLM Hybrid Review

DAY 3 · 6차시

15:00-15:50 (쉬는 시간 17:30-18:00)

정적 분석·LLM Hybrid Review

Static analysis · Unit test · hybrid_review.json

진행

정적 분석·LLM Hybrid Review
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

AGENTS.md
tests/test_course_services.py

확인할 결과

hybrid_review.json

6차시 입력과 결과

입력	————	baseline_review.json
이번 차시	————	정적 분석·LLM Hybrid Review
결과	————	hybrid_review.json
다음 연결	————	Precision·Recall 기반 평가

정적 분석·LLM Hybrid Review 필요성

문법·unused import까지 LLM에 맡기면 느리고 일관성도 낮다.

ruff에서 hybrid_review.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: hybrid_review.json

정적 분석·LLM Hybrid Review 학습 결과

- 01 — ruff·pytest 결과와 의미 리뷰를 한 context로 합친다.
- 02 — 실행하지 않은 test를 passed라고 prompt에 써 둔다. 왜 위험한지 설명한다.
- 03 — hybrid_review.json에서 정상과 실패 상태를 직접 확인한다.

정적 분석·LLM Hybrid Review 용어

용어	이 수업에서 쓰는 뜻
Static analysis	코드를 실행하지 않고 규칙으로 검사
Unit test	작은 기능의 동작을 검증
Hybrid review	결정론 검사와 의미 검토 결합
Evidence	실제 명령과 변경 라인

hybrid_review.json 입출력

INPUT

AGENTS.md

OUTPUT

hybrid_review.json

PROCESS

ruff → focused pytest → diff parse

VERIFY

결정론 finding과 test evidence가 같은 report에 존재한다.
test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

정적 분석·LLM Hybrid Review 실행 흐름

01

ruff

02

focused pytest

03

diff parse

04

LLM review

05

deduplicate

마지막 판단은 hybrid_review.json에서 사람이 확인합니다.

hybrid_review.json 정상·실패 계약

정상	결정론 finding과 test evidence가 같은 report에 존재한다.
경계	test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.
외부 쓰기	정적 분석·LLM Hybrid Review: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·hybrid_review.json

정적 분석·LLM Hybrid Review 파일 지도

입력·fixture

AGENTS.md

구현 코드

tests/test_course_services.py

검증·test

src/course_services/review_service.py

따라하기 notebook

.github/workflows/test.yml

정적 분석·LLM Hybrid Review 개념·코드

Static analysis

코드를 실행하지 않고 규칙으로 검사

```
AGENTS.md
```

Unit test

작은 기능의 동작을 검증

```
tests/test_course_services.py
```

Hybrid review

결정론 검사와 의미 검토 결합

```
src/course_services/review_service.py
```

Evidence

실제 명령과 변경 라인

```
.github/workflows/test.yml
```


정적 분석·LLM Hybrid Review 선택 기준

구분	역할	이번 차시의 판단 기준
Static analysis	ruff 단계의 입력 기준	결정론 test로 먼저 확인
Unit test	focused pytest 단계의 교체 경계	adapter 경계를 확인
Hybrid review	diff parse 단계의 운영 조건	비용·지연·권한을 확인
Evidence	LLM review 단계의 판단 기록	사람 판단과 운영 기록을 확인

정적 분석·LLM Hybrid Review 대표 실패

실패

실행하지 않은 test를 passed라고 prompt에 써 둔다.

사용자 영향

실행하지 않은 test를 passed라고 prompt에 써 둔다. 상태에서는 hybrid_review.json을 운영 판단에 사용할 수 없다.

정적 분석·LLM Hybrid Review 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

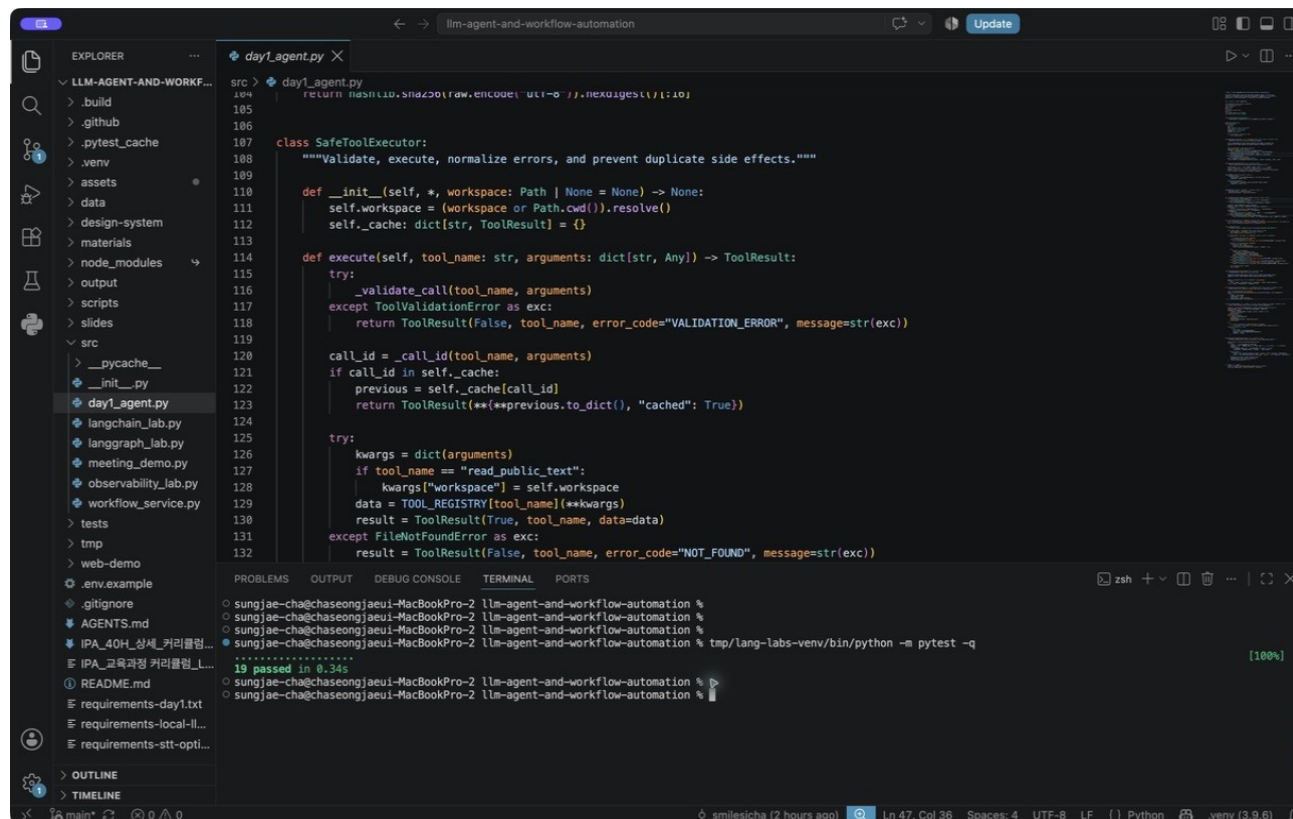
정적 분석·LLM Hybrid Review 복구 경로

- 01 — 탐지: test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.
- 02 — 중단: 실행하지 않은 test를 passed라고 prompt에 써 둔다.
- 03 — 복구: command·exit code·stdout 요약을 실행 후 evidence bundle에 넣는다.
- 04 — 증거: hybrid_review.json과 test 결과를 함께 남긴다.

정적 분석·LLM Hybrid Review Codex 검증 루프

tests/test_course_services.py
의 책임을 찾고
hybrid_review.json 계약을
focused test로 고정합니다.

정적 분석·LLM Hybrid Review
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



정적 분석·LLM Hybrid Review Codex 작업 명세

목표

코드를 만든 뒤 **focused test**와 **full suite**를 실제로 실행하고 결과를 보고하게 한다.

허용 경로

tests/test_course_services.py
src/course_services/review_service.py

완료 조건

결정론 finding과 test evidence가 같은 report에 존재한다.
test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

금지

정적 분석·LLM Hybrid Review: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

정적 분석·LLM Hybrid Review 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | 정적 분석·LLM Hybrid Review 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 결정론 finding과 test evidence가 같은 report에 존재한다. |
| 03 | Claude review | —— | hybrid_review.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 정적 분석·LLM Hybrid Review의 두 LLM 의견과 test는 자동 승인이 아니다 |

정적 분석·LLM Hybrid Review 정상 시연

- 01 — 열기: AGENTS.md
- 02 — 구현 확인: tests/test_course_services.py
- 03 — 실행: python -m pytest -q tests/test_course_services.py
- 04 — 성공 신호: 결론론 finding과 test evidence가 같은 report에 존재한다.

정적 분석·LLM Hybrid Review 실행 명령

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

정적 분석·LLM Hybrid Review: repository root · Python 3.12 · .env 미출력

06_hybrid_review.json 실행 결과

```
{
  "static_findings": [
    {
      "path": "src/payment_job.py",
      "line": 9,
      "severity": "P0",
      "title": "검증되지 않은 문자열이 코드로 실행될 수 있음",
      "body": "입력 경로에 따라 임의 코드 실행으로 이어질 수 있습니다.",
      "evidence": "result = eval(command)",
      "suggestion": "허용된 명령이나 파서만 호출하는 명시적 registry로 교체하세요.",
      "confidence": 0.99,
      "rule_id": "unsafe-dynamic-execution"
    },
    {
      "path": "src/payment_job.py",
      "line": 11,
      "severity": "P1",
      "title": "외부 쓰기 전에 사람 승인 경계가 보이지 않음",
      "body": "재시도나 잘못된 대상 선택이 실제 게시·수정으로 이어질 수 있습니다.",
      "evidence": "requests.post(\"https://example.invalid/jobs\", json=payload)",
      "suggestion": "dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.",
      "confidence": 0.91,
      "rule_id": "external-write-without-approval"
    },
    ...
  ]
}
```

정적 분석·LLM Hybrid Review 실패·복구 시연

재현

test 미실행은
READY_FOR_HUMAN_MERGE가
아니다.

복구

command·exit code·stdout 요약을
실행 후 evidence bundle에 넣는다.

확인: test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

정적 분석·LLM Hybrid Review 소프트웨어 제작

열 파일

tests/test_course_services.py
src/course_services/review_service.py

작업 범위

정적 분석·LLM Hybrid Review 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

결정론 finding과 test evidence가 같은 report에 존재한다.
hybrid_review.json 저장

정적 분석·LLM Hybrid Review Notebook 셀

materials/day3/day3_service_lab.ipynb · 6차시

```
test_evidence = run_command(sys.executable, "-m", "pytest", "-q", "tests/test_course_services.py", "-k", "unified_diff
or maps_findings")
hybrid_review = {
    "static_findings": baseline["findings"],
    "test_evidence": test_evidence,
    "llm_adapter": {"requested": False, "reason": "DETERMINISTIC_BASELINE_FIRST"},
    "ready_for_evaluation": test_evidence["returncode"] == 0,
}
assert hybrid_review["ready_for_evaluation"] is True
save_json("06_hybrid_review.json", hybrid_review)
{"finding_count": len(hybrid_review["static_findings"]), "test_returncode": test_evidence["returncode"]}
```

정적 분석·LLM Hybrid Review Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py
```

저장 위치

```
output/course-labs/day3/06_hybrid_review.json
```

결정론 finding과 test evidence가 같은 report에 존재한다.
실패 시 hybrid_review.json의 status·error_code를 먼저 확인

정적 분석·LLM Hybrid Review 실패 증거

test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

```
result_file = "output/course-labs/day3/06_hybrid_review.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

hybrid_review.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

정적 분석·LLM Hybrid Review 정상 Case

NORMAL

결정론 finding과 test evidence가 같은 report에 존재한다.

- 01 — AGENTS.md 입력과 command가 기록됐는가
- 02 — hybrid_review.json schema가 validate되는가
- 03 — 정적 분석·LLM Hybrid Review focused test가 실제로 통과했는가

정적 분석·LLM Hybrid Review 경계 Case

BOUNDARY

test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

- 01 — 정적 분석·LLM Hybrid Review: raw traceback 대신 stable error_code가 남는가
- 02 — hybrid_review.json: 외부 쓰기·자동 메일이 false인가
- 03 — test 미실행은 READY_FOR_HUMAN_MERGE가 아니다. 재현이 가능한가

정적 분석·LLM Hybrid Review 현업 적용

PR이 열리면 자동 검사 후 개발자가 고위험 finding만 검토

- 01 — 정적 분석·LLM Hybrid Review은 합성·비식별 sample로 먼저 실행
- 02 — PR이 열리면 자동 검사 후 개발자가 고위험 finding만 검토의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — hybrid_review.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — hybrid_review.json을 운영 검토 자료로 사용

정적 분석·LLM Hybrid Review 포트폴리오 적용

정적 분석·LLM Hybrid Review · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 실행하지 않은 test를 passed라고 prompt에 써 둔다.의 사용자 영향을 한 문장으로 설명
- 02 — 정적 분석·LLM Hybrid Review 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 코드를 만든 뒤 focused test와 full suite를 실제로 실행하고 결과를 보고하게 한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — hybrid_review.json과 README 재현 절차를 제출

정적 분석·LLM Hybrid Review 운영 점검

품질

무엇을 결정론 finding과 test evidence가 같은 report에 존재한다.로 정의하는가

복구

실행하지 않은 test를 passed라고 prompt에 써 둔다.
발생 시 hybrid_review.json에서 원인 상태를 확인

안전

정적 분석·LLM Hybrid Review의 사람 승인과
external_write=false 위치

관측

hybrid_review.json에서 어떤 status·error_code를
보는가

hybrid_review.json 실행 증거

- 01 — 설명: 정적 분석·LLM Hybrid Review이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: hybrid_review.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Precision·Recall 기반 평가

Precision·Recall 기반 평가

Precision · Recall · review_eval.json

진행

Precision·Recall 기반 평가
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/eval_service.py
data/day5_eval/
golden_review_findings.json

확인할 결과

review_eval.json

7차시 입력과 결과

입력	hybrid_review.json
이번 차시	Precision·Recall 기반 평가
결과	review_eval.json
다음 연결	오류 분석과 회귀 방지

Precision·Recall 기반 평가 필요성

많이 지적하는 reviewer가 실제로 더 좋은 reviewer라는 보장은 없다.

key 정규화에서 review_eval.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: review_eval.json

Precision·Recall 기반 평가 학습 결과

- 01 — golden finding과 predicted finding을 exact key로 비교한다.
- 02 — 제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다. 왜 위험한지 설명한다.
- 03 — review_eval.json에서 정상과 실패 상태를 직접 확인한다.

Precision·Recall 기반 평가 용어

용어	이 수업에서 쓰는 뜻
Precision	제기한 문제 중 맞은 비율
Recall	잡아야 할 문제 중 찾은 비율
F1	precision과 recall의 조화 평균
Golden set	합의된 기대 finding 모음

review_eval.json 입출력

INPUT

src/course_services/eval_service.py

OUTPUT

review_eval.json

PROCESS

key 정규화 → 교집합 → false positive

VERIFY

fixture case precision·recall·F1이 1.0이다.
recall 또는 precision 0.8 미만이면 HOLD다.

Precision-Recall 기반 평가 실행 흐름

01

key 정규화

02

교집합

03

false positive

04

false negative

05

release gate

마지막 판단은 review_eval.json에서 사람이 확인합니다.

review_eval.json 정상·실패 계약

정상	—— fixture case precision·recall·F1이 1.0이다.
경계	—— recall 또는 precision 0.8 미만이면 HOLD다.
외부 쓰기	—— Precision·Recall 기반 평가: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·review_eval.json

Precision·Recall 기반 평가 파일 지도

입력·fixture

`src/course_services/eval_service.py`

구현 코드

`data/day5_eval/golden_review_findings.json`

검증·test

`tests/test_course_services.py`

따라하기 notebook

`materials/day3/day3_service_lab.ipynb`

Precision·Recall 기반 평가 개념·코드

Precision

제기한 문제 중 맞은 비율

```
src/course_services/eval_service.py
```

Recall

잡아야 할 문제 중 찾은 비율

```
data/day5_eval/golden_review_findings.json
```

F1

precision과 recall의 조화 평균

```
tests/test_course_services.py
```

Golden set

합의된 기대 finding 모음

```
materials/day3/day3_service_lab.ipynb
```

Precision·Recall 기반 평가 선택 기준

구분	역할	이번 차시의 판단 기준
Precision	key 정규화 단계의 입력 기준	결정론 test로 먼저 확인
Recall	교집합 단계의 교체 경계	adapter 경계를 확인
F1	false positive 단계의 운영 조건	비용·지연·권한을 확인
Golden set	false negative 단계의 판단 기록	사람 판단과 운영 기록을 확인

Precision·Recall 기반 평가 대표 실패

실패

제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다.

사용자 영향

제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다. 상태에서는 review_eval.json을 운영 판단에 사용할 수 없다.

Precision·Recall 기반 평가 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

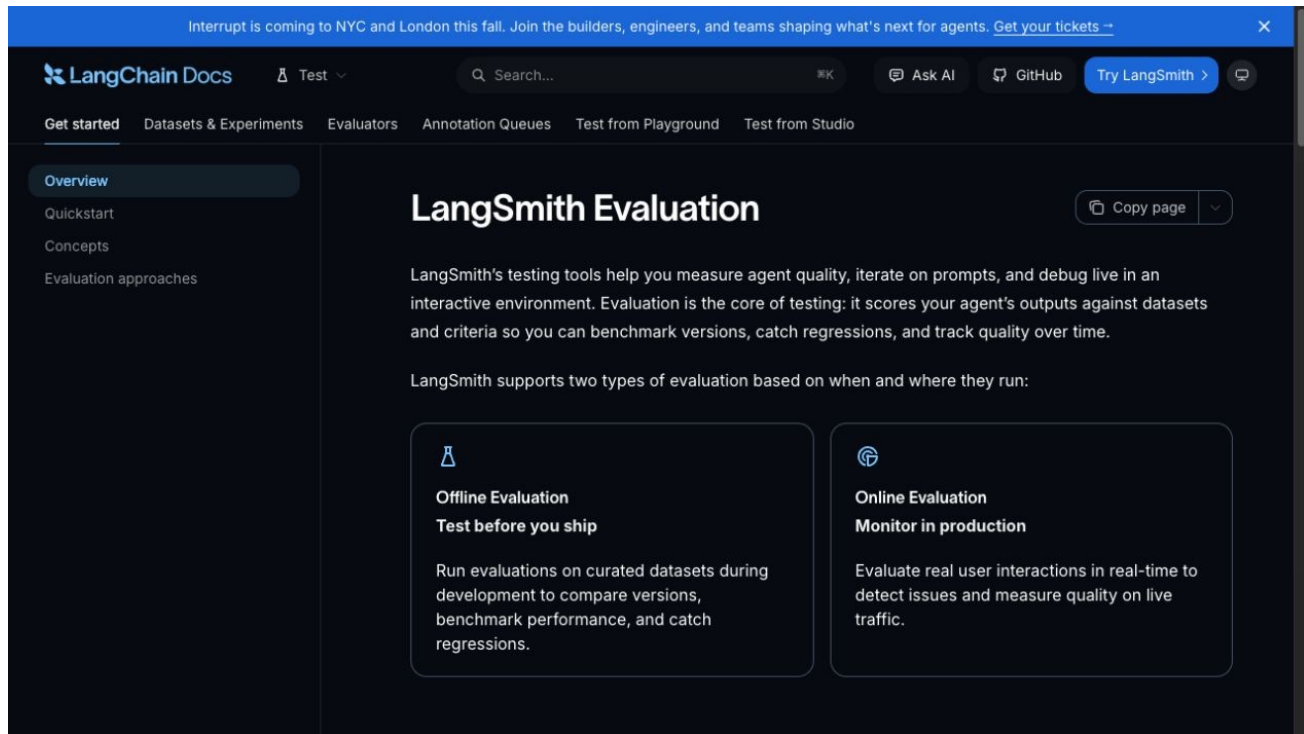
Precision·Recall 기반 평가 복구 경로

- 01 — 탐지: recall 또는 precision 0.8 미만이면 HOLD다.
- 02 — 중단: 제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다.
- 03 — 복구: path·line·rule_id 같은 안정 key를 평가에 사용한다.
- 04 — 증거: review_eval.json과 test 결과를 함께 남긴다.

Precision·Recall 기반 평가 Codex 검증 루프

data/day5_eval/
golden_review_findings.json의
책임을 찾고 review_eval.json
계약을 focused test로 고정합니다.

Precision·Recall 기반 평가
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Precision·Recall 기반 평가 Codex 작업 명세

목표

평가 함수를 구현할 때 empty prediction·empty expected 경계를 test한다.

허용 경로

data/day5_eval/golden_review_findings.json
tests/test_course_services.py

완료 조건

fixture case precision·recall·F1이 1.0이다.
recall 또는 precision 0.8 미만이면 HOLD다.

금지

Precision·Recall 기반 평가: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Precision·Recall 기반 평가 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | Precision·Recall 기반 평가 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | fixture case precision·recall·F1이 1.00이다. |
| 03 | Claude review | —— | review_eval.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Precision·Recall 기반 평가의 두 LLM 의견과 test는 자동 승인이 아니다 |

Precision·Recall 기반 평가 정상 시연

- 01 — 열기: `src/course_services/eval_service.py`
- 02 — 구현 확인: `data/day5_eval/golden_review_findings.json`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 04 — 성공 신호: fixture case precision·recall·F1이 1.0이다.

Precision-Recall 기반 평가 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k offline_eval
```

실행 전 확인

Precision-Recall 기반 평가: repository root · Python 3.12 · .env 미출력

07_review_eval.json 실행 결과

```
{
  "metrics": {
    "true_positive": 3,
    "false_positive": 0,
    "false_negative": 0,
    "precision": 1,
    "recall": 1,
    "f1": 1
  },
  "release_gate": {
    "decision": "READY",
    "reasons": [],
    "human_release_required": true
  }
}
```

Precision·Recall 기반 평가 실패·복구 시연

재현

recall 또는 precision 0.8 미만이면 HOLD
다.

복구

path·line·rule_id 같은 안정 key를
평가에 사용한다.

확인: recall 또는 precision 0.8 미만이면 HOLD다.

Precision·Recall 기반 평가 소프트웨어 제작

열 파일 `data/day5_eval/golden_review_findings.json`
`tests/test_course_services.py`

작업 범위 Precision·Recall 기반 평가 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 fixture case precision·recall·F1이 1.00이다.
`review_eval.json` 저장

Precision-Recall 기반 평가 Notebook 셀

materials/day3/day3_service_lab.ipynb · 7차시

```
from src.course_services.eval_service import evaluate_review_findings, release_gate

expected = json.loads((ROOT / "data/day5_eval/golden_review_findings.json").read_text(encoding="utf-8"))
review_metrics = evaluate_review_findings(baseline["findings"], expected)
review_gate = release_gate(review_metrics=review_metrics, safety_passed=True, latency_seconds=0.2)
assert review_gate["decision"] == "READY"
save_json("07_review_eval.json", {"metrics": review_metrics, "release_gate": review_gate})
{"precision": review_metrics["precision"], "recall": review_metrics["recall"], "f1": review_metrics["f1"]}
```

Precision·Recall 기반 평가 Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k offline_eval
```

저장 위치

```
output/course-labs/day3/07_review_eval.json
```

fixture case precision·recall·F1이 1.0이다.
실패 시 review_eval.json의 status·error_code를 먼저 확인

Precision·Recall 기반 평가 실패 증거

recall 또는 precision 0.8 미만이면 HOLD다.

```
result_file = "output/course-labs/day3/07_review_eval.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

review_eval.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Precision·Recall 기반 평가 정상 Case

NORMAL

fixture case precision·recall·F1이 1.0이다.

- 01 — src/course_services/eval_service.py 입력과 command가 기록됐는가
- 02 — review_eval.json schema가 validate되는가
- 03 — Precision·Recall 기반 평가 focused test가 실제로 통과했는가

Precision·Recall 기반 평가 경계 Case

BOUNDARY

recall 또는 precision 0.8 미만이면 HOLD다.

- 01 — Precision·Recall 기반 평가: raw traceback 대신 stable error_code가 남는가
- 02 — review_eval.json: 외부 쓰기·자동 메일이 false인가
- 03 — recall 또는 precision 0.8 미만이면 HOLD다. 재현이 가능한가

Precision·Recall 기반 평가 현업 적용

모델 교체 전 기존 reviewer보다 나아졌는지 정량 비교

- 01 — Precision·Recall 기반 평가은 합성·비식별 sample로 먼저 실행
- 02 — 모델 교체 전 기존 reviewer보다 나아졌는지 정량 비교의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — review_eval.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_eval.json을 운영 검토 자료로 사용

Precision·Recall 기반 평가 포트폴리오 적용

Precision·Recall 기반 평가 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— 제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다.의 사용자 영향을 한 문장으로 설명
- 02 ——— Precision·Recall 기반 평가 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— 평가 함수를 구현할 때 empty prediction·empty expected 경계를 test한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— review_eval.json과 README 재현 절차를 제출

Precision·Recall 기반 평가 운영 점검

품질

무엇을 fixture case precision·recall·F1이 1.0이다.로 정의하는가

복구

제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다. 발생 시 review_eval.json에서 원인 상태를 확인

안전

Precision·Recall 기반 평가의 사람 승인과 external_write=false 위치

관측

review_eval.json에서 어떤 status·error_code를 보는가

review_eval.json 실행 증거

- 01 — 설명: Precision·Recall 기반 평가이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 03 — 증거: review_eval.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 오류 분석과 회귀 방지

DAY 3 · 8차시

16:40-17:30 (쉬는 시간 17:30-18:00)

오류 분석과 회귀 방지

Error analysis · Regression · day3_review_report.md

진행

오류 분석과 회귀 방지
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/
codex_harness.py
tests/test_course_services.py

확인할 결과

day3_review_report.md

8차시 입력과 결과

입력	————	review_eval.json
이번 차시	————	오류 분석과 회귀 방지
결과	————	day3_review_report.md
다음 연결	————	17:30-18:00 쉬는 시간

오류 분석과 회귀 방지 필요성

평균 점수만 보면 특정 언어·파일·위험 유형의 반복 실패가 보이지 않는다.

실패 목록에서 day3_review_report.md까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: day3_review_report.md

오류 분석과 회귀 방지 학습 결과

- 01 — false positive·false negative 원인을 분류하고 Day 3 report를 만든다.
- 02 — 점수를 올리려고 기존 assertion이나 policy를 약화한다. 왜 위험한지 설명한다.
- 03 — day3_review_report.md에서 정상과 실패 상태를 직접 확인한다.

오류 분석과 회귀 방지 용어

용어	이 수업에서 쓰는 뜻
Error analysis	틀린 결과의 패턴을 분류
Regression	이전보다 나빠진 동작
Ablation	구성요소를 빼 영향 확인
Human merge	최종 반영은 사람이 결정

day3_review_report.md 입출력

INPUT

src/course_services/codex_harness.py

OUTPUT

day3_review_report.md

PROCESS

실패 목록 → 원인 label → 최소 수정

VERIFY

허용된 path·focused test·diff review가 모두 충족된다.
.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD
다.

오류 분석과 회귀 방지 실행 흐름

01

실패 목록

02

원인 label

03

최소 수정

04

회귀 test

05

commit-tag

마지막 판단은 day3_review_report.md에서 사람이 확인합니다.

day3_review_report.md 정상·실패 계약

정상	허용된 path·focused test·diff review가 모두 충족된다.
경계	.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
외부 쓰기	오류 분석과 회귀 방지: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·day3_review_report.md

오류 분석과 회귀 방지 파일 지도

입력·fixture

src/course_services/codex_harness.py

구현 코드

tests/test_course_services.py

검증·test

AGENTS.md

따라하기 notebook

materials/day3/day3_service_lab.ipynb

오류 분석과 회귀 방지 개념·코드

Error analysis

틀린 결과의 패턴을 분류

```
src/course_services/codex_harness.py
```

Regression

이전보다 나빠진 동작

```
tests/test_course_services.py
```

Ablation

구성요소를 빼 영향 확인

```
AGENTS.md
```

Human merge

최종 반영은 사람이 결정

```
materials/day3/day3_service_lab.ipynb
```

오류 분석과 회귀 방지 선택 기준

구분	역할	이번 차시의 판단 기준
Error analysis	실패 목록 단계의 입력 기준	결정론 test로 먼저 확인
Regression	원인 label 단계의 교체 경계	adapter 경계를 확인
Ablation	최소 수정 단계의 운영 조건	비용·지연·권한을 확인
Human merge	회귀 test 단계의 판단 기록	사람 판단과 운영 기록을 확인

오류 분석과 회귀 방지 대표 실패

실패

점수를 올리려고 기존 `assertion`이나 `policy`를 약화한다.

사용자 영향

점수를 올리려고 기존 `assertion`이나 `policy`를 약화한다. 상태에서는 `day3_review_report.md`을 운영 판단에 사용할 수 없다.

오류 분석과 회귀 방지 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

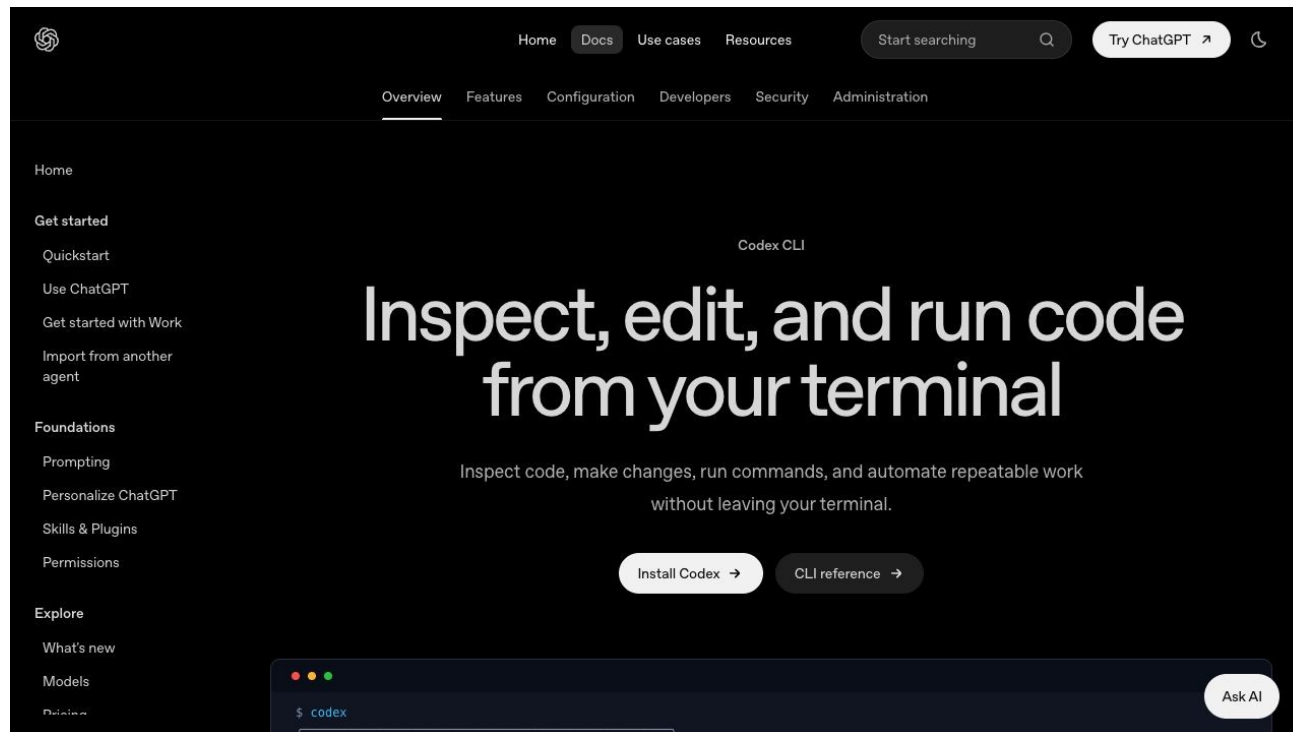
오류 분석과 회귀 방지 복구 경로

- 01 — 탐지: .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
- 02 — 중단: 점수를 올리려고 기존 assertion이나 policy를 약화한다.
- 03 — 복구: 실패를 대표 test로 추가하고 최소 변경만 허용한다.
- 04 — 증거: day3_review_report.md과 test 결과를 함께 남긴다.

오류 분석과 회귀 방지 Codex 검증 루프

`tests/test_course_services.py`
의 책임을 찾고
`day3_review_report.md` 계약을
`focused test`로 고정합니다.

오류 분석과 회귀 방지
저장소 이해 → 허용 범위 변경
→ `focused test` → `diff` 리뷰
→ 사람 merge



오류 분석과 회귀 방지 Codex 작업 명세

목표

diff를 검토하고 actionable finding만 보고한 뒤 test 증거를 다시 확인한다.

허용 경로

tests/test_course_services.py
AGENTS.md

완료 조건

허용된 path·focused test·diff review가 모두 충족된다.
.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

금지

오류 분석과 회귀 방지: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

오류 분석과 회귀 방지 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | 오류 분석과 회귀 방지 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 허용된 path·focused test·diff review가 모두 충족된다. |
| 03 | Claude review | —— | day3_review_report.md 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 오류 분석과 회귀 방지의 두 LLM 의견과 test는 자동 승인이 아니다 |

오류 분석과 회귀 방지 정상 시연

- 01 — 열기: `src/course_services/codex_harness.py`
- 02 — 구현 확인: `tests/test_course_services.py`
- 03 — 실행: `git diff -- src/course_services tests/test_course_services.py`
- 04 — 성공 신호: 허용된 `path·focused test·diff review`가 모두 충족된다.

오류 분석과 회귀 방지 실행 명령

```
$ git diff -- src/course_services tests/test_course_services.py
```

실행 전 확인

오류 분석과 회귀 방지: repository root · Python 3.12 · .env 미출력

08_day3_report.json 실행 결과

```
{
  "codex_task": "목표\nreview rule 하나와 정상·실패 test를 추가한다.\n\n먼저 읽을 파일\n- AGENTS.md\n- README.md\n\n변경 허용 범위\n- src/course_services\n- tests\n\n완료 조건\n- python -m pytest -q tests/test_course_services.py\n\n금지 행동\n- secret 출력 또는 commit\n- workspace 밖 파일 변경\n- 사람 승인 없는 외부 쓰기\n\n통과를 위한 기존 test 약화\n\n복구 지점\n- 작업 전 commit\n\n마지막에 변경 파일, 핵심 diff, 실행한 test 명령과 결과를 보고하세요.",
  "ready_assessment": {
    "decision": "READY_FOR_HUMAN_MERGE",
    "reasons": [],
    "unexpected_paths": [],
    "missing_tests": [],
    "human_merge_required": true
  },
  "hold_assessment": {
    "decision": "HOLD",
    "reasons": [
      "UNEXPECTED_CHANGED_PATH",
      "ACCEPTANCE_TEST_NOT_PASSED",
      "DIFF_REVIEW_REQUIRED",
      "SECRET_DETECTED"
    ],
    "unexpected_paths": [
      ".env"
    ],
    "missing_tests": [
      "python -m pytest -q tests/test_course_services.py"
    ],
    "human_merge_required": true
  },
  ...
}
```

오류 분석과 회귀 방지 실패·복구 시연

재현

.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

복구

실패를 대표 test로 추가하고 최소 변경만 허용한다.

확인: .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

오류 분석과 회귀 방지 소프트웨어 제작

열 파일

tests/test_course_services.py
AGENTS.md

작업 범위

오류 분석과 회귀 방지 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

허용된 path·focused test·diff review가 모두 충족된다.
day3_review_report.md 저장

오류 분석과 회귀 방지 Notebook 셀

materials/day3/day3_service_lab.ipynb · 8차시

```
from src.course_services.codex_harness import CodexTaskSpec, assess_codex_run, render_codex_task
from src.course_services.course_demo import build_course_demo

spec = CodexTaskSpec(
    objective="review rule 하나와 정상·실패 test를 추가한다.",
    allowed_paths=("src/course_services", "tests"),
    acceptance_tests=("python -m pytest -q tests/test_course_services.py",),
)
ready_assessment = assess_codex_run(
    spec,
    changed_paths=["src/course_services/review_service.py", "tests/test_course_services.py"],
    executed_tests={"python -m pytest -q tests/test_course_services.py": True},
    diff_reviewed=True, secrets_detected=False,
)
hold_assessment = assess_codex_run(
    spec, changed_paths=[".env"], executed_tests={}, diff_reviewed=False, secrets_detected=True,
)
scorecard = build_course_demo(3, workspace_root=R00T)
day3_result = {
    "codex_task": render_codex_task(spec), "ready_assessment": ready_assessment,
    # ... 다음 줄은 Notebook에서 계속
```


오류 분석과 회귀 방지 Terminal 실행

```
$ git diff -- src/course_services tests/test_course_services.py
```

저장 위치

```
output/course-labs/day3/08_day3_report.json
```

허용된 path·focused test·diff review가 모두 충족된다.
실패 시 day3_review_report.md의 status·error_code를 먼저 확인

오류 분석과 회귀 방지 실패 증거

.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

```
result_file = "output/course-labs/day3/08_day3_report.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

day3_review_report.md이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

오류 분석과 회귀 방지 정상 Case

NORMAL

허용된 path·focused test·diff review가 모두 충족된다.

- 01 — src/course_services/codex_harness.py 입력과 command가 기록됐는가
- 02 — day3_review_report.md schema가 validate되는가
- 03 — 오류 분석과 회귀 방지 focused test가 실제로 통과했는가

오류 분석과 회귀 방지 경계 Case

BOUNDARY

.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

- 01 — 오류 분석과 회귀 방지: raw traceback 대신 stable error_code가 남는가
- 02 — day3_review_report.md: 외부 쓰기·자동 메일이 false인가
- 03 — .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다. 재현이 가능한가

오류 분석과 회귀 방지 현업 적용

현업은 merge 보조, 구직자는 리뷰 품질 리포트와 회귀 test를 포트폴리오로 제출

- 01 — 오류 분석과 회귀 방지는 합성·비식별 sample로 먼저 실행
- 02 — 현업은 merge 보조, 구직자는 리뷰 품질 리포트와 회귀 test를 포트폴리오로 제출의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — day3_review_report.md 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — day3_review_report.md을 운영 검토 자료로 사용

오류 분석과 회귀 방지 포트폴리오 적용

오류 분석과 회귀 방지 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 점수를 올리려고 기존 assertion이나 policy를 약화한다.의 사용자 영향을 한 문장으로 설명
- 02 — 오류 분석과 회귀 방지 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — diff를 검토하고 actionable finding만 보고한 뒤 test 증거를 다시 확인한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — day3_review_report.md과 README 재현 절차를 제출

오류 분석과 회귀 방지 운영 점검

품질

무엇을 허용된 path·focused test·diff review가 모두 충족된다.로 정의하는가

복구

점수를 올리려고 기존 assertion이나 policy를 약화한다.
발생 시 day3_review_report.md에서 원인 상태를 확인

안전

오류 분석과 회귀 방지의 사람 승인과
external_write=false 위치

관측

day3_review_report.md에서 어떤
status·error_code를 보는가

day3_review_report.md 실행 증거

- 01 — 설명: 오류 분석과 회귀 방지이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git diff -- src/course_services tests/test_course_services.py`
- 03 — 증거: day3_review_report.md과 정상·실패 test 결과가 남아 있다.

17:30-18:00 쉬는 시간